

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Крымский федеральный университет имени В. И. Вернадского»

Физико-технический институт
кафедра компьютерной инженерии и моделирования

Чудопалов Богдан Андреевич

ПЛАТФОРМА УПРАВЛЕНИЯ ПРОГРАММНЫМИ ПРОЕКТАМИ

выпускная квалификационная работа (уровень бакалавриата)

направление подготовки: 09.03.01 «Информатика и вычислительная техника»

Научный руководитель
доцент кафедры компьютерной
инженерии и моделирования,
к.т.н.

_____ Парменов О.И.
(подпись, дата)

К ЗАЩИТЕ ДОПУСКАЮ
Заведующий кафедрой
компьютерной инженерии и
моделирования

_____ Милюков В.В.
(подпись, дата)

Симферополь 2026

Чудопалов, Б.А. Разработка платформы управления программными проектами: выпускн. квалиф. работа: 09.03.01. – Симферополь: КФУ им. В.И. Вернадского, 2026. – 55 с., 28 ил., 3 ист., 9 табл.

Объект исследования: процесс автоматизации жизненного цикла программного обеспечения

Предмет исследования: разработка платформы управления программными проектами, которая поддерживает автоматическую сборку, тестирование и доставку на сервер.

Методы исследования: анализ существующих решений CI/CD решений, изучение технологий контейнеризации, проектирование архитектуры взаимодействия между компонентами платформы, анализ эффективности платформы путем сравнения процессов развертывания вручную и с использованием платформы.

Цель работы: создание платформы, реализующей автоматизацию сборки, тестирования, проверки качества кода и доставки на сервера для программного проекта.

Разработана платформа, включающая автоматическую систему сборки, тестирования и доставки программных проектов. Платформа поддерживает такие языки программирования, как Java, C#, Python. Проведено тестирование временных затрат на развертывание проекта.

DEVOPS, CI/CD, DOCKER, SPRING, ANSIBLE, PIPELINES, NEXUS

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
ГЛАВА 1 Теоретические основы DevOps-автоматизации и управления процессами разработки программного обеспечения.....	8
1.1 Жизненный цикл ПО и автоматизация процессов жизненного цикла	8
1.2 Анализ существующих решений автоматизации жизненного цикла...	10
1.3 Функциональность существующих решений и их недостатки.....	13
1.4 Постановка задачи исследования.....	15
ГЛАВА 2 Практическая реализация платформы.....	17
2.1 Общая архитектура и компоненты платформы.....	17
2.2 Серверная часть и база данных.....	20
2.2.1 Аутентификация и авторизация.....	23
2.2.2 Контейнеризация бэкенда.....	23
2.3 Клиентская часть.....	25
2.4 Подсистема мониторинга.....	28
2.5 Компоненты обеспечения качества и хранения артефактов.....	30
2.6 GitLab CE и GitLab Runner.....	34
2.7 Проверка работы платформы.....	36
ГЛАВА 3 Тестирование платформы.....	40
3.1 Сравнение ручного и автоматизированного развертывания.....	40
3.2 Анализ работы платформы в аварийных ситуациях.....	46
3.3 Варианты улучшения и развития платформы.....	50
ЗАКЛЮЧЕНИЕ.....	54
ЛИТЕРАТУРА.....	55

СПИСОК ИСПОЛЬЗУЕМЫХ ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

CI/CD — практика автоматизации разработки ПО, включающая непрерывную интеграцию (Continuous Integration) и непрерывную доставку/развертывание (Continuous Delivery/Deployment), где изменения кода регулярно объединяются, тестируются и автоматически доставляются в рабочую среду.

Deploy (деплой) - развертывание и запуск веб-приложения или сайта в его рабочей среде, то есть на сервере или хостинге

Pipeline (пайплайн) — это детализированная и стандартизированная последовательность этапов

Master-Slave (Controller-Agent) — архитектурная модель, в которой один центральный узел (Master, контроллер) управляет процессами, распределяет задачи и контролирует их выполнение, а подчинённые узлы (Slave, агенты) выполняют назначенные операции и передают результаты обратно мастеру, обеспечивая централизованное управление и масштабируемое распределение вычислительных ресурсов.

Dependency hell — ситуация в разработке ПО, когда управление внешними библиотеками и пакетами становится крайне сложным из-за конфликтов версий, несовместимости и взаимозависимостей, что приводит к ошибкам при установке, сборке или запуске программ.

GitLab Runner — приложение, которое используется вместе с GitLab CI/CD для выполнения задач (jobs) в конвейере (pipeline). Оно подключается к GitLab, получает задания из .gitlab-ci.yml и запускает их на доступной вычислительной инфраструктуре.

ВВЕДЕНИЕ

Коммерческая разработка программного обеспечения характеризуется необходимостью быстрого выпуска обновлений и быстрой доставки на продуктовые контуры, чтобы конечные пользователи как можно быстрее получали новый функционал. Исходя из этого повышается потребность в инструментах, которые обеспечивают автоматизацию процессов, затрагивающих доставку в продуктовую среду. В этих условиях большую роль начинают играть платформы, которые объединяют процессы DevOps и предоставляющие единую экосистему для управления программным кодом.

Развитие технологий контейнеризации, автоматизации и CI/CD позволяет перейти от ручных операций к системам, которые способны автоматически собирать, тестировать и разворачивать приложения, тем самым снижая вероятность ошибок и повышая эффективность работы команды разработчиков. Преимуществом подобных платформ также является то, что они создают условия для стандартизации процессов разработки и развертывание программных продуктов.

Работа посвящена разработке платформы управления программными проектами, обеспечивающей автоматизированную сборку, тестирование, проверку качества кода и развертывание на серверах с поддержкой контейнерных технологий. Актуальность темы обусловлена необходимостью оптимизации и унификации процессов разработки приложений любого масштаба и назначения. Когда речь идет о разработке приложений без использования подходов DevOps и автоматизации, команды вынуждены выполнять большое количество ручных операций, связанных с подготовкой окружения, сборкой проекта, запуском тестов, анализом кода и последующим развертыванием приложений. Очевидно, что такие процессы требуют больших затрат времени, увеличивают вероятность возникновения ошибок и существенно замедляют выпуск обновлений. В результате продукт

становится менее конкурентоспособным, резко возрастает стоимость его сопровождения, а стабильность приложения напрямую зависит от человеческого фактора.

В рамках работы реализована платформа, объединяющая функциональность автоматизированной сборки, тестирования и развертывания (деплоя). Платформа использует самостоятельно размещенные экземпляры GitLab и Nexus для хранения исходного кода приложения, интеграцию с репозиториями, запуск конвейеров сборки и тестирования.

Целью исследования является разработка и внедрение платформы, обеспечивающей полный цикл автоматизации управления программными проектами от хранения исходного кода до его развертывания.

Объектом исследования выступает процесс автоматизации жизненного цикла программного обеспечения, а предметом — реализация платформы управления программными проектами с поддержкой CI/CD, мониторинга, деплоя и анализа качества кода.

В ходе работы проводится анализ существующих технологий CI/CD и DevOps-платформ, реализация прототипа на основе современных инструментов контейнеризации и мониторинга, а также тестирование эффективности автоматизированных процессов.

Практическая значимость работы заключается в создании платформы, которая может применяться для автоматизации разработки в малых и средних командах, обеспечивая сокращение времени выпуска программного продукта, уменьшение количества ошибок и повышение прозрачности всех этапов разработки. Полученные результаты могут быть использованы при разработке аналогичных DevOps-решений, а также в исследовательских

задачах, связанных с управлением жизненным циклом программного обеспечения.

Работа состоит из трёх глав, введения, заключения и списка литературы. Первая глава посвящена теоретическим основам DevOps-методологий, CI/CD и автоматизации жизненного цикла программных проектов. Вторая глава описывает архитектуру и практическую реализацию разработанной платформы. Третья глава включает анализ работы платформы, её тестирование и оценку полученного эффекта от автоматизации.

ГЛАВА 1 Теоретические основы DevOps-автоматизации и управления процессами разработки программного обеспечения

1.1 Жизненный цикл ПО и автоматизация процессов жизненного цикла

Жизненный цикл программного обеспечения (SDLC — Software Development Life Cycle) представляет собой совокупность этапов, которые проходит программный продукт от момента принятия решения о его создании до полного прекращения эксплуатации [1]. Классическая модель жизненного цикла включает в себя следующие основные фазы: сбор и анализ требований, проектирование архитектуры, написание программного кода, тестирование, развертывание и техническая поддержка.

В традиционных моделях разработки (например, каскадной) переход между этапами осуществляется последовательно, что зачастую приводит к длительным циклам релиза и позднему обнаружению ошибок. С развитием гибких подходов DevOps акцент сместился на цикличность, частые итерации и автоматизацию рутинных процессов.

DevOps (Development Operations) — это методология, направленная на активное взаимодействие специалистов по разработке и специалистов по информационно-технологическому обслуживанию, а также на взаимную интеграцию их рабочих процессов друг в друга для обеспечения качества продукта [2].

Ключевым инструментом реализации DevOps-подхода является автоматизация процессов CI/CD:

- Continuous Integration (CI, Непрерывная интеграция) — практика разработки, заключающаяся в частом слиянии рабочих копий кода в общую основную ветку разработки. Этот процесс сопровождается автоматическим запуском сборок проекта и выполнением тестов для максимально быстрого выявления ошибок интеграции.

- Continuous Delivery (CD, Непрерывная доставка) — расширение CI, при котором изменения не только собираются и тестируются, но и автоматически подготавливаются к выпуску в производственную среду. Код всегда находится в состоянии, готовом к релизу, но само развертывание выполняется вручную по решению команды.
- Continuous Deployment (CD, Непрерывное развертывание) — дальнейшее развитие практики Continuous Delivery, при котором каждое успешно прошедшее тестирование изменение автоматически разворачивается в производственной среде без участия человека. Это обеспечивает максимально быструю доставку функционала пользователям.

Процесс автоматизации жизненного цикла обычно реализуется через конвейеры (pipelines). Пайплайн описывает последовательность шагов, которые должен пройти код.

Типовой процесс автоматизированного жизненного цикла выглядит следующим образом:

1. Commit: Разработчик отправляет код в систему контроля версий (VCS).
2. Build: Система CI автоматически собирает приложение.
3. Test: Запуск автоматических тестов и анализаторов кода.
4. Release: Сохранение артефактов сборки (бинарных файлов, образов) в репозиторий (например, Nexus).
5. Deploy: Автоматическая доставка приложения на тестовый или продуктовый контур.

Современный CI/CD процесс отображен в виде схемы на рисунке 1.

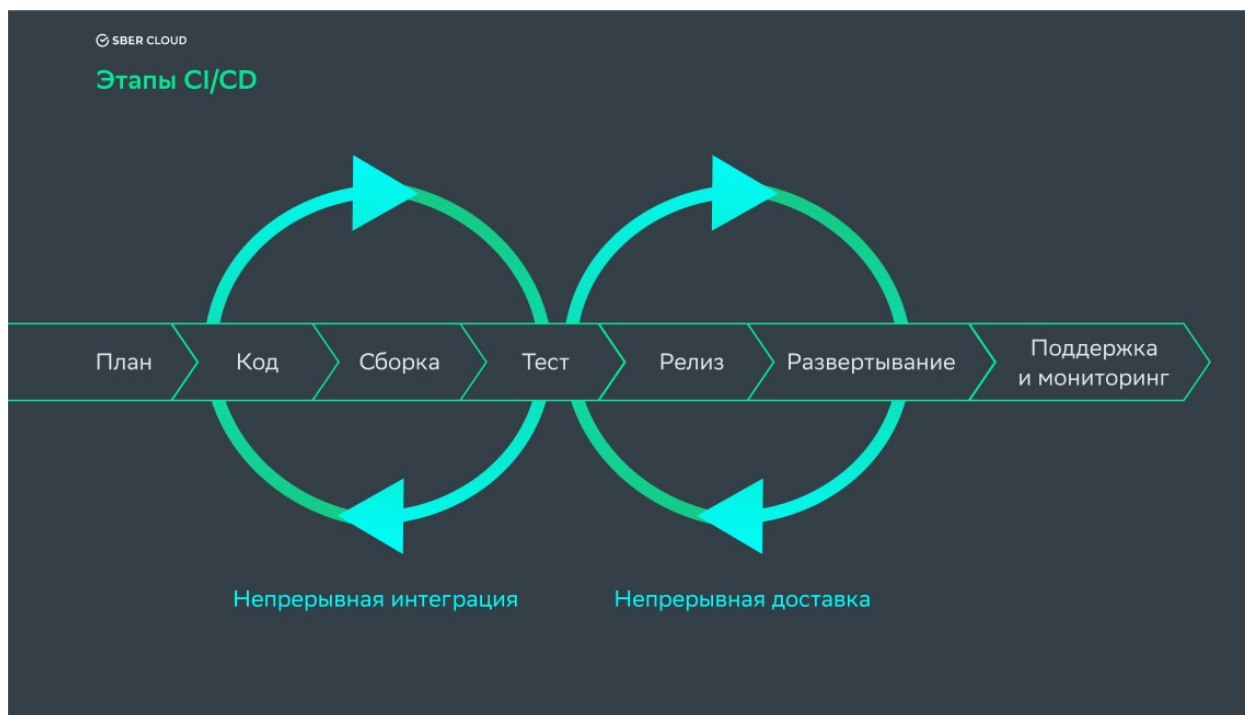


Рисунок 1 — Этапы CI/CD

Автоматизация процессов позволяет сократить время выхода на рынок, снизить влияние человеческого фактора при развертывании и повысить стабильность систем. Однако для реализации такого подхода необходимы специализированные программные комплексы.

1.2 Анализ существующих решений автоматизации жизненного цикла

На рынке существует множество инструментов для реализации CI/CD, которые различаются архитектурой, способом конфигурации и степенью интеграции с системами контроля версий. Рассмотрим наиболее популярные решения, используемые в индустрии: Jenkins, Gitlab CI/CD и GitHub Actions.

Jenkins — это одна из старейших и наиболее распространенных систем автоматизации с открытым исходным кодом, написанная на Java (рисунок 2). Основное преимущество Jenkins заключается в его расширяемости: функциональность обеспечивается тысячами плагинов, поддерживаемых сообществом [3]. Ее специфика:

Архитектура: Использует модель Master-Slave (Controller-Agent), где мастер-узлы управляют расписанием и распределением задач, а агенты выполняют непосредственную сборку и тестирование.

Конфигурация: Пайплайны описываются с использованием Groovy-скриптов, которые могут быть декларативными или скриптовыми.

Особенности: Высокая гибкость, возможность запуска на собственной инфраструктуре, но сложная настройка и обслуживание - dependency hell.

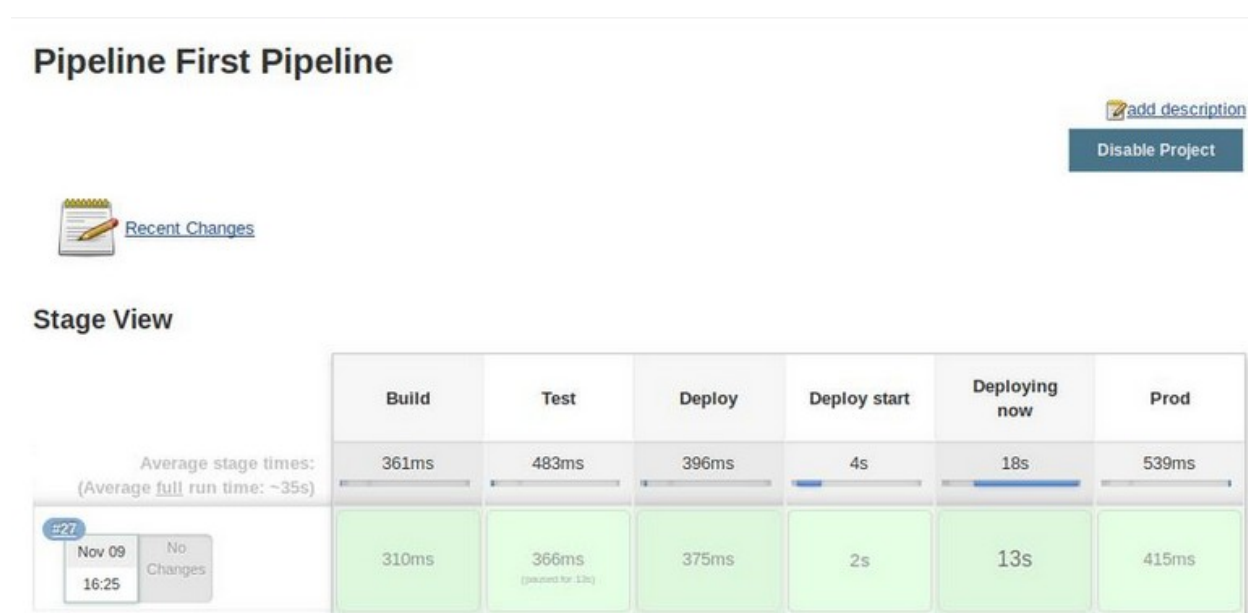


Рисунок 2 — Пайплайн в Jenkins

Другое комплексное решение, которое объединяет репозиторий кода, трекер задач и систему CI/CD в одном интерфейсе - GitLab CI/CD, который является частью платформы GitLab (рисунок 3).

Архитектура: Использует концепцию GitLab Runners — агентов, которые устанавливаются на сервера и выполняют задачи, полученные от координатора GitLab.

Конфигурация: Настройка производится через файл `.gitlab-ci.yml`, расположенный в корне репозитория проекта. Используется синтаксис YAML.

Особенности: Тесная интеграция с репозиторием, встроенный реестр контейнеров, поддержка Auto DevOps. Является одним из стандартов для Self-hosted решений в корпоративном секторе.

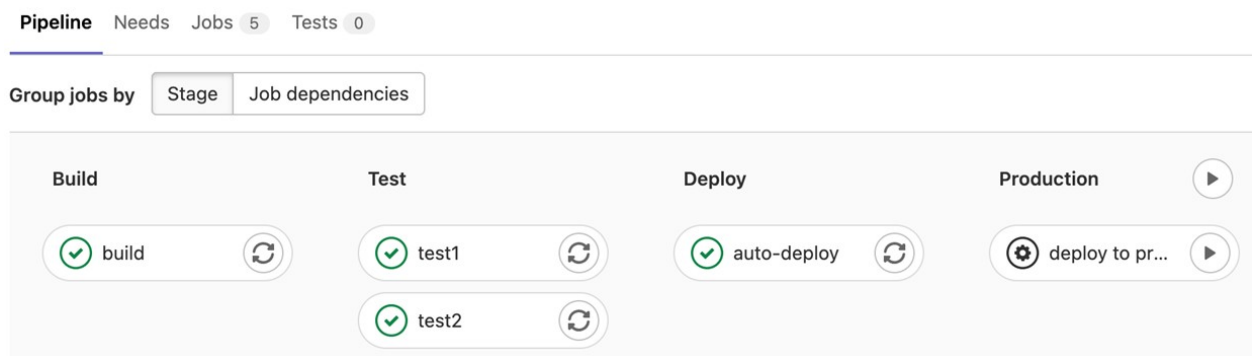


Рисунок 3 — Пайплайн в GitLab CI/CD

GitHub Actions - относительно новый инструмент от компании GitHub, который позволяет автоматизировать рабочие процессы непосредственно в репозитории GitHub (рисунок 4).

Архитектура: Облачная инфраструктура, предоставляемая GitHub или собственные раннеры.

Конфигурация: Используются YAML-файлы, расположенные в директории `.github/workflows`.

Особенности: Огромный маркетплейс готовых действий (Actions), которые можно переиспользовать в своих пайплайнах как "кирпичики". Отлично подходит для Open Source проектов, но имеет ограничения в бесплатной версии для частных репозиториев.

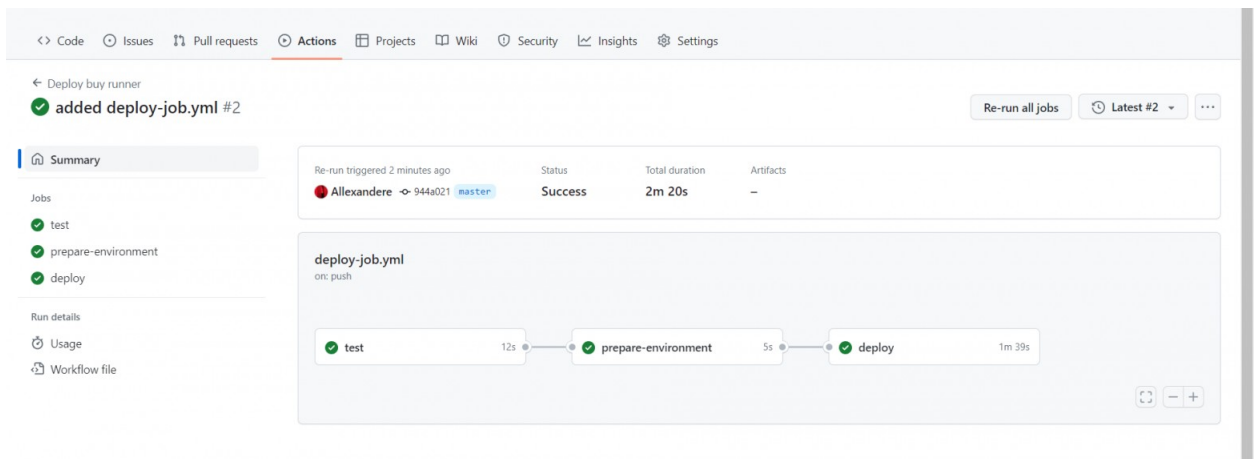


Рисунок 4 — Пайплайн в GitHub Actions

1.3 Функциональность существующих решений и их недостатки

Сравнение функциональности существующих CI/CD-систем приведено в таблице 1. В некоторых системах часть задач полностью автоматизированы, в других - требуют ручной настройки через стандартные инструменты [2].

Отметим, что все доступные системы имеют ограничение: они не автоматизируют процесс настройки пайплайнов [2]. Разработчику или DevOps-инженеру приходится вручную создавать репозитории, подключать интеграции, копировать шаблоны и задавать переменные. Это замедляет запуск проектов и усложняет стандартизацию процессов.

Фактически, в них отсутствует полноценная экосистема, которая автоматизирует инициализацию проекта, централизованно управляет шаблонами и ролями, настраивает интеграции и переменные, обеспечивает стандартизацию процессов и минимизирует зависимость работы от узких специалистов. Из-за этого страдают как быстрота запуска новых проектов, так и сохранение единообразия CI/CD-пайплайнов.

Таблица 1 - Сравнение функциональных возможностей CI/CD-систем и предлагаемой платформы

Аспект сравнения (функционал)	GitLab	GitHub Actions	Jenkins	имеется ли реальная практическая потребность в данном функционале
автоматическое создание репозитория	нет	нет	нет	да
генерация CI/CD-файла по стеку	нет	возможна с ручной настройкой	нет	да
автоматическая настройка интеграций	нет	нет	возможна через shared libraries, требует настройки	да
Self-hosted развёртывание	да	нет	да	да
подходит для команд без DevOps-специалиста	нет	частично	нет	да

Таким образом, проведённый анализ показывает, что современные CI/CD-системы хорошо справляются с выполнением сборки, тестирования и деплоя приложений [2]. Однако все они требуют ручной настройки репозитория, шаблонов и интеграций, что создаёт дополнительные временные затраты, увеличивает нагрузку на разработчика и делает процессы менее стандартизированными.

Но именно эти аспекты особо актуальны для малых и средних команд, стартапов и образовательных проектов, где важны скорость запуска новых проектов, стандартизация пайплайнов и минимальная зависимость от узких специалистов.

1.4 Постановка задачи исследования

Проведенный анализ показал, что, несмотря на наличие развитых инструментов CI/CD, процесс их первоначальной настройки и интеграции остается трудоемким. Существует разрыв между потребностью бизнеса в быстром старте проектов и технической сложностью организации DevOps-конвейеров. Ручная настройка репозиторий, написание конфигурационных файлов «с нуля» и управление доступами приводят к ошибкам, фрагментации процессов и отсутствию стандартизации.

В связи с этим основной целью работы является проектирование и программная реализация платформы управления программными проектами, выступающей в роли оркестратора над существующим CI/CD-движком. Платформа должна скрывать сложность инфраструктурной настройки от разработчика, предоставляя единый интерфейс для управления жизненным циклом приложения.

Для достижения поставленной цели необходимо решить следующий комплекс задач:

1. Проектирование архитектуры платформы: разработка структуры взаимодействия компонентов системы, включающей пользовательский интерфейс, серверную часть для обработки логики, базу данных для хранения метаданных о проектах и модули интеграции с внешними сервисами.

2. Разработка механизма шаблонизации и генерации конфигураций: создание подсистемы, способной на основе выбранного технологического стека автоматически генерировать корректные файлы конфигурации CI/CD и скрипты развертывания.

3. Автоматизация взаимодействия с системой контроля версий: реализация программного интерфейса для взаимодействия с API GitLab с

целью автоматического создания репозитория, настройки веток, управления правами доступа и внедрения необходимых переменных окружения.

4. Реализация централизованного управления зависимостями и артефактами: обеспечение интеграции с репозиторием артефактов для хранения результатов сборки.

5. Создание пользовательского интерфейса: разработка веб-интерфейса, позволяющего пользователям создавать новые проекты, выбирать необходимый стек для разработки проекта.

ГЛАВА 2 Практическая реализация платформы

2.1 Общая архитектура и компоненты платформы

Для обоснования выбора технологий опишем общую архитектуру платформы. При проектировании платформы были приняты следующие архитектурные решения:

1. Контейнеризация всех компонентов. Каждый компонент инфраструктуры — от базы данных до системы мониторинга — развернуты в Docker-контейнерах. Преимущества данного подхода заключаются в том, что обеспечивается воспроизводимость среды, упрощается развертывание на конечных серверах и устраняется проблема несоответствия зависимостей. Оркестрация контейнеров выполняется с помощью Docker Compose, позволяющего декларативно описать инфраструктуру в одном файле.

2. Клиент серверная архитектура с REST API. Фронтенд взаимодействует с бэкенд-сервером посредством HTTP-запросов, обмениваясь данными в JSON-формате. Это обеспечивает слабую связность между клиентской и серверной частями, позволяя внедрять новый функционал быстрее.

3. Многослойная архитектура бэкенда. Серверная часть спроектирована по принципу layered architecture с разделением на слои представления, бизнеслогики, доступа к данным. Зависимости между слоями направлены строго сверху вниз, что обеспечивает тестируемость и сопровождение кода.

Архитектура платформы отображена на рисунке 5.

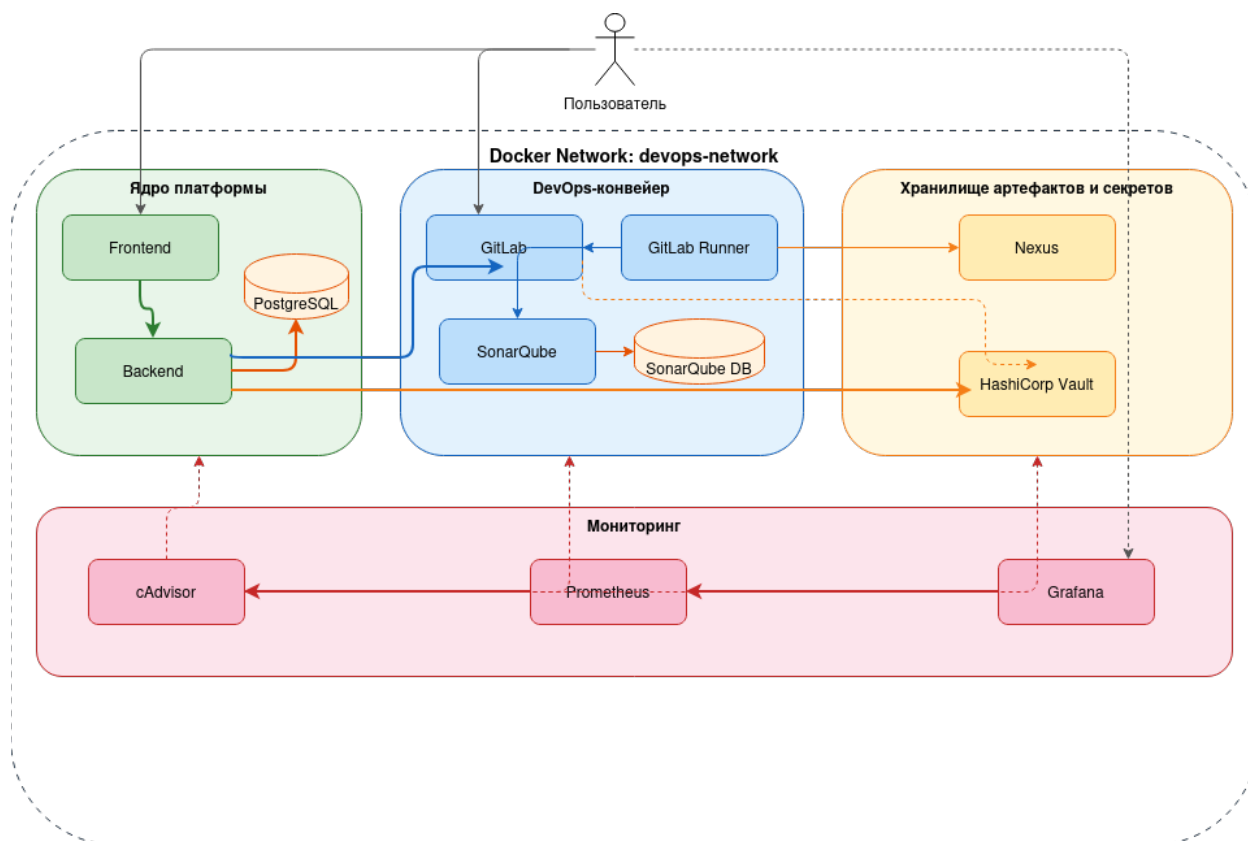


Рисунок 5 — Общая архитектура платформы

Компоненты можно логически поделить на четыре группы, каждая из которых будет подробно рассмотрена далее:

- ядро платформы - фронтенд, бэкенд и база данных, реализующие основную бизнес-логику;
- DevOps инструментарий - GitLab, GitLab Runner и SonarQube, обеспечивающие CI/CD;
- хранилище артефактов и секретов - Nexus и HashiCorp Vault;
- подсистема мониторинга - cAdvisor, Prometheus и Grafana.

Контейнеры платформы объединены в единую пользовательскую Docker-сеть devops-network с драйвером bridge. Все данные, требующие сохранения между перезапусками контейнеров, хранятся в именованных Docker-томах. Именованные тома управляются Docker Engine и сохраняются на файловой системе хоста независимо от жизненного цикла контейнеров, что

гарантирует сохранность данных при обновлении, перезапуске или пересоздании контейнеров.

Для стабильной работы платформы реализованы четыре механизма обеспечения надёжности:

1. Политика перезапуска. Контейнеры сконфигурированы с политикой `restart: unless-stopped`. Это означает, что контейнер автоматически перезапускается при аварийном завершении или при перезагрузке хост-системы, но не будет перезапущен, если был остановлен вручную.

2. Проверки работоспособности (`healthcheck`). Критичные контейнеры оснащены механизмами проверки:

- PostgreSQL проверяется утилитой `pg_isready` каждые 10 секунд с допуском 5 попыток;
- бэкенд проверяется HTTP-запросом к эндпоинту `/actuator/health` каждые 30 секунд.

Результаты `healthcheck` используются Docker Compose для определения готовности сервиса.

3. Условные зависимости при запуске. Docker Compose обеспечивает корректный порядок старта: бэкенд запускается только после прохождения `healthcheck` базой данных; фронтенд — после готовности бэкенда. Это исключает ситуации, когда приложение пытается подключиться к ещё не запущенному сервису.

4. Выделение ресурсов. Контейнеру GitLab назначен увеличенный объём разделяемой памяти, что предотвращает ошибки при работе встроенной базы данных PostgreSQL, используемой GitLab для внутренних нужд.

2.2 Серверная часть и база данных

Серверная часть платформы реализована на языке Java версии 17 с использованием фреймворка Spring Boot 3.2.1. Выбор данного стека обусловлен несколькими факторами: зрелая экосистема Spring предоставляет готовые решения для работы с базами данных (Spring Data JPA), обеспечения безопасности (Spring Security), построения REST API (Spring Web) и реактивного взаимодействия с внешними сервисами (Spring WebFlux). Сборка проекта осуществляется с помощью Apache Maven, а управление зависимостями декларативно описано в файле pom.xml. Ключевые зависимости проекта приведены в таблице 2.

Таблица 2 - Основные зависимости Java реализации

Зависимость	Назначение
spring-boot-starter-web	Встроенный веб-сервер Tomcat, поддержка REST
spring-boot-starter-data-jpa	ORM-доступ к базе данных через Hibernate
spring-boot-starter-security	Аутентификация и авторизация
spring-boot-starter-validation	Валидация входных данных
spring-boot-starter-webflux	HTTP-клиент WebClient для вызовов внешних API
spring-boot-starter-actuator	Эндпоинты мониторинга и проверки состояния
postgresql	JDBC-драйвер PostgreSQL
jwt (0.12.3)	Генерация и верификация JWT-токенов
lombok (1.18.30)	Сокращение шаблонного кода (аннотации)

Схема базы данных платформы состоит из четырёх таблиц, связанных между собой отношениями «один ко многим» (рисунок 6).

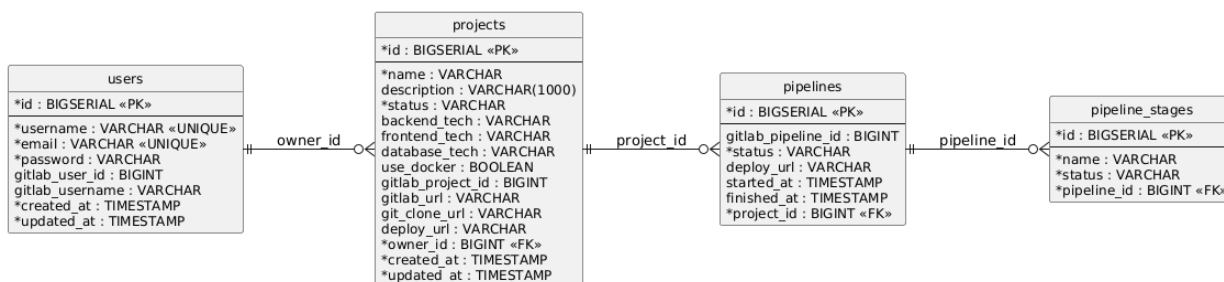


Рисунок 6 — ER-диаграмма базы данных

В работе реализована многослойная архитектура бэкенда: серверная часть спроектирована по принципу layered architecture и организована в виде пакетной структуры; зависимости между слоями направлены строго сверху вниз: контроллеры вызывают сервисы, сервисы обращаются к репозиториям.

Таблица 3 - Пакетная структура серверной части

Пакет	Слой	Назначение
controller	Представление	REST-контроллеры, обработка HTTP-запросов
service	Бизнес-логика	Сервисы с основной логикой приложения
repository	Доступ к данным	JPA-репозитории для работы с БД
entity	Доменная модель	JPA-сущности и перечисления
dto.request	Передача данных	Объекты входящих запросов с валидацией
dto.response	Передача данных	Объекты исходящих ответов
config	Конфигурация	Настройки Security
exception	Обработка ошибок	Глобальный обработчик исключений

Взаимодействие клиентской части с сервером осуществляется через REST API; эндпоинты сгруппированы в три контроллера, каждый из которых отвечает за свою предметную область (таблица 4).

Таблица 4 - REST API эндпоинты платформы

Метод	Путь	Назначение	Авторизация
POST	/api/auth/register	Регистрация пользователя	Нет
POST	/api/auth/login	Вход в систему	Нет
GET	/api/auth/me	Получение текущего пользователя	JWT
PUT	/api/auth/profile	Обновление профиля	JWT
PUT	/api/auth/password	Смена пароля	JWT
GET	/api/projects	Список проектов пользователя	JWT
GET	/api/projects/{id}	Информация о проекте	JWT
POST	/api/projects	Создание нового проекта	JWT
DELETE	/api/projects/{id}	Удаление проекта	JWT
GET	/api/projects/{id}/gitlab	Ссылки на GitLab-репозиторий	JWT

Запросы и ответы передаются в формате JSON. Входящие данные проходят валидацию с помощью аннотаций Jakarta Validation, определённых в DTO-классах запросов. Например, при создании проекта проверяется, что название содержит только допустимые символы и имеет длину от 2 до 100 символов. В случае нарушения ограничений клиенту возвращается ответ с HTTP-статусом 400 и описанием ошибки. Обработка исключений

централизована в классе `GlobalExceptionHandler`, перехватывающем все типы ошибок и формирующем унифицированный формат ответа через обёртку `ApiResponse`

2.2.1 Аутентификация и авторизация

Безопасность платформы реализована на основе JSON Web Token (JWT) и Spring Security. При отключённом управлении сессиями (stateless-режим) каждый запрос должен содержать токен в заголовке `Authorization` в формате `Bearer <token>`. Механизм аутентификации работает следующим образом.

При регистрации или входе сервис `JwtService` генерирует токен, подписанный секретным ключом алгоритмом HMAC-SHA256. Время жизни токена составляет 24 часа. Токен содержит email пользователя в поле `subject`, время создания и время истечения.

При каждом входящем запросе фильтр `JwtAuthenticationFilter`, встроенный в цепочку фильтров Spring Security перед стандартным `UsernamePasswordAuthenticationFilter`, извлекает токен из заголовка, проверяет его подпись и срок действия, после чего загружает данные пользователя из базы и устанавливает контекст аутентификации. Если токен отсутствует или невалиден, запрос передаётся дальше без аутентификации, и при попытке доступа к защищённому ресурсу возвращается HTTP-статус 401.

2.2.2 Контейнеризация бэкенда

Для контейнеризации серверной части (рисунок 7) применяется многоступенчатая сборка Docker (multi-stage build). На первом этапе (build) используется образ `maven:3.9-eclipse-temurin-17-alpine`: сначала копируется файл `pom.xml` и загружаются зависимости, что позволяет кэшировать данный слой при последующих сборках, затем копируются исходные файлы и

выполняется компиляция. На втором этапе используется облегчённый образ `eclipse-temurin:17-jre-alpine`, содержащий только среду выполнения Java. Итоговый JAR-файл копируется из первого этапа, а приложение запускается от имени непривилегированного пользователя `spring`, что повышает безопасность контейнера. Такой подход позволяет сократить размер итогового образа, исключив из него компилятор, Maven и исходные файлы проекта.

```
# =====
# Stage 1: Build
# =====
FROM maven:3.9-eclipse-temurin-17-alpine AS build

WORKDIR /app

COPY pom.xml .
RUN mvn dependency:go-offline -B

COPY src ./src
RUN mvn clean package -DskipTests

# =====
# Stage 2: Runtime
# =====
FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring

COPY --from=build /app/target/*.jar app.jar

EXPOSE 8080

HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 \
  CMD wget -qO- http://localhost:8080/api/actuator/health || exit 1

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Рисунок 7 — Dockerfile для бэкенда платформы

2.3 Клиентская часть

Клиентская часть платформы реализована на фреймворке Vue.js версии 3. В качестве сборщика используется Vite 5, обеспечивающий быструю горячую перезагрузку модулей при разработке и оптимизированную сборку для продакшена. Взаимодействие с REST API бэкенда осуществляется через HTTP-клиент Axios.

Взаимодействие с бэкендом осуществляется посредством HTTP-запросов к серверной части проходят через единый экземпляр Axios, сконфигурированный в модуле `api.js`. Базовый URL установлен как относительный путь `/api`, что позволяет использовать один и тот же код как при локальной разработке, так и в контейнерном окружении.

Для экземпляра Axios настроены два перехватчика. Перехватчик запросов автоматически извлекает JWT-токен из `localStorage` и добавляет его в заголовок `Authorization` в формате `Bearer <token>`. Перехватчик ответов отслеживает HTTP-статус 401 (Unauthorized): при его получении токен удаляется из хранилища, а пользователь перенаправляется на страницу входа. Такой централизованный подход исключает необходимость обработки аутентификации в каждом компоненте.

На основе базового экземпляра Axios реализованы два сервисных модуля: `authService` — методы регистрации, входа, получения текущего пользователя, обновления профиля и смены пароля; `projectService` — методы получения списка проектов, получения проекта по идентификатору, создания и удаления проекта. Каждый метод возвращает промис с данными ответа, что позволяет использовать их в хранилищах `Pinia` и компонентах через конструкцию `async/await`.

Корневой компонент `App.vue` определяет общую структуру приложения: боковая панель навигации `Navbar` отображается только для аутентифицированных пользователей, компонент уведомлений `Alert`

присутствует на всех страницах, а область содержимого отрисовывается компонентом router-view.

Для контейнеризации клиентской части применяется многоступенчатая сборка Docker, аналогичная подходу, использованному для бэкенда (рисунок 8). На первом этапе используется образ `node:20-alpine`: устанавливаются зависимости командой `npm ci` и выполняется сборка приложения командой `npm run build`, результатом которой является каталог `dist` с оптимизированными статическими файлами. На втором этапе используется образ `nginx:alpine`, в который копируются конфигурация Nginx и собранное приложение.

```
# =====
# Stage 1: Build
# =====
FROM node:20-alpine AS build

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

# =====
# Stage 2: Runtime (Nginx)
# =====
FROM nginx:alpine

RUN rm /etc/nginx/conf.d/default.conf

COPY nginx.conf /etc/nginx/conf.d/default.conf

COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80

HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
  CMD wget -qO- http://localhost/ || exit 1

CMD ["nginx", "-g", "daemon off;"]
```

Рисунок 8 - Dockerfile для клиентской части

Конфигурация Nginx решает несколько задач. Первая — раздача статических файлов SPA-приложения.

Вторая задача — проксирование запросов к бэкенду. Все запросы с префиксом /api/ перенаправляются на контейнер бэкенда по внутреннему DNS-имени backend:8080. При проксировании передаются заголовки X-Real-IP, X-Forwarded-For и X-Forwarded-Proto, что позволяет бэкенду определять реальный адрес клиента и используемый протокол. Таймаут чтения установлен в 300 секунд для поддержки длительных операций, таких как запуск CI/CD пайплайнов.

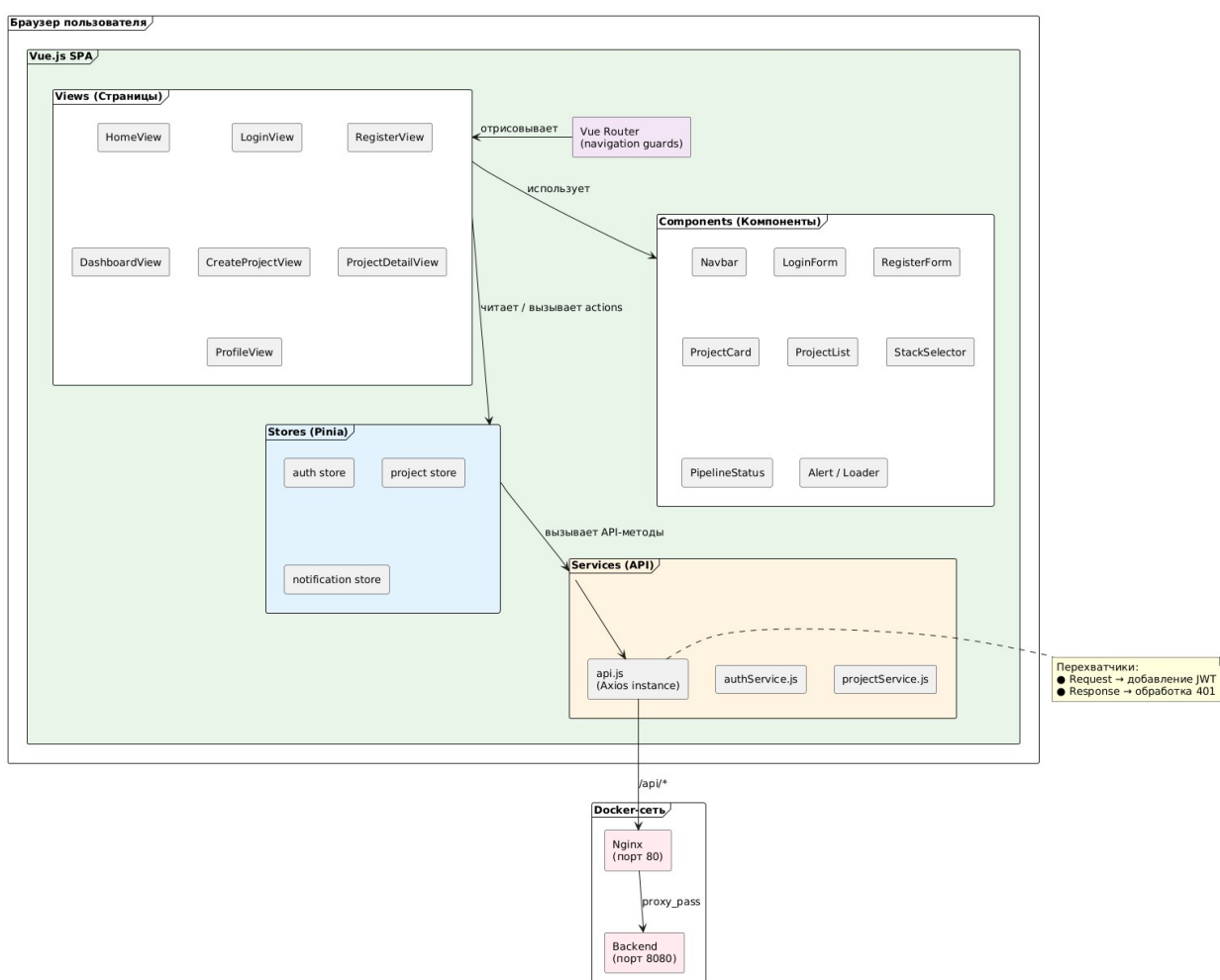


Рисунок 9 — Общая архитектура клиентской части платформы

2.4 Подсистема мониторинга

Для обеспечения контроля состояния компонентов платформы реализована подсистема мониторинга, построенная по модели pull-based сбора метрик. Данная модель предполагает, что центральный сервер мониторинга самостоятельно опрашивает источники данных с заданным интервалом, в отличие от push-based модели, при которой агенты отправляют метрики на сервер. Подсистема состоит из трёх компонентов (рисунок 10): cAdvisor осуществляет сбор метрик контейнеров, Prometheus агрегирует и хранит временные ряды, Grafana визуализирует данные в виде интерактивных дашбордов.

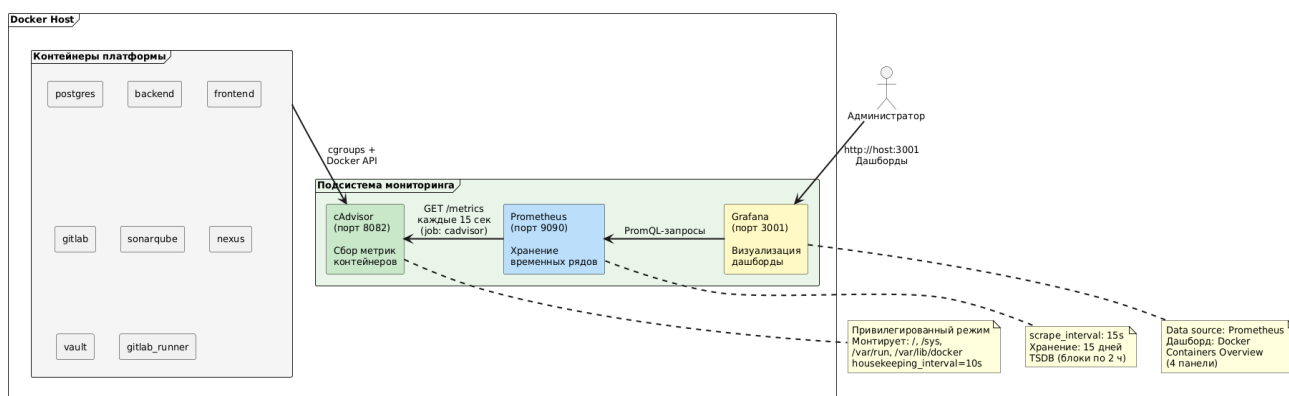


Рисунок 10 — Общая архитектура подсистемы мониторинга

Сбор метрик контейнеров - Container Advisor (cAdvisor) — инструмент с открытым исходным кодом, разработанный компанией Google для мониторинга ресурсов контейнеров. В отличие от внешних агентов, cAdvisor работает непосредственно с подсистемой cgroups ядра Linux, что обеспечивает минимальные накладные расходы и высокую точность данных. Описание ключевых метрик представлено в таблице 5.

Таблица 5 - Категории метрик, собираемые cAdvisor

Категория	Ключевые метрики	Описание
Процессор	container_cpu_usage_seconds_total	Суммарное время использования CPU в секундах
Память	container_memory_usage_bytes	Текущее потребление оперативной памяти в байтах
Сеть	container_network_receive_bytes_total, container_network_transmit_bytes_total	Суммарный объём принятых и переданных данных
Диск	container_fs_reads_bytes_total, container_fs_writes_bytes_total	Суммарный объём операций чтения и записи

Для агрегации и хранения метрик использован Prometheus, который выполняет роль центрального хранилища временных рядов. Его конфигурация определена в файле prometheus.yml (рисунок 11); он монтируется из каталога monitoring/prometheus/ хост-машины. Конфигурация содержит два параметра глобального уровня и два задания на сбор метрик.

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']

  - job_name: 'prometheus'
    static_configs:
      - targets: ['prometheus:9090']
```

Рисунок 11 — Конфигурация Prometheus

Для визуализации данных использована Grafana, которая предоставляет веб-интерфейс для визуализации метрик, собранных Prometheus. Контейнер

Grafana сконфигурирован с учётными данными администратора по умолчанию и доступен на порту 3001 хост-машины. Данные дашбордов и настройки сохраняются в именованном Docker-томе grafana-data, что обеспечивает их сохранность при перезапуске контейнера. Grafana запускается после Prometheus, что гарантирует доступность источника данных к моменту первого запроса.

В качестве источника данных подключён Prometheus по внутреннему адресу `http://prometheus:9090`. Для платформы разработан дашборд «`Docker Containers Overview`», содержащий четыре панели, каждая из которых отображает ключевую категорию метрик (пример с одним графиком представлен на рисунке 12).

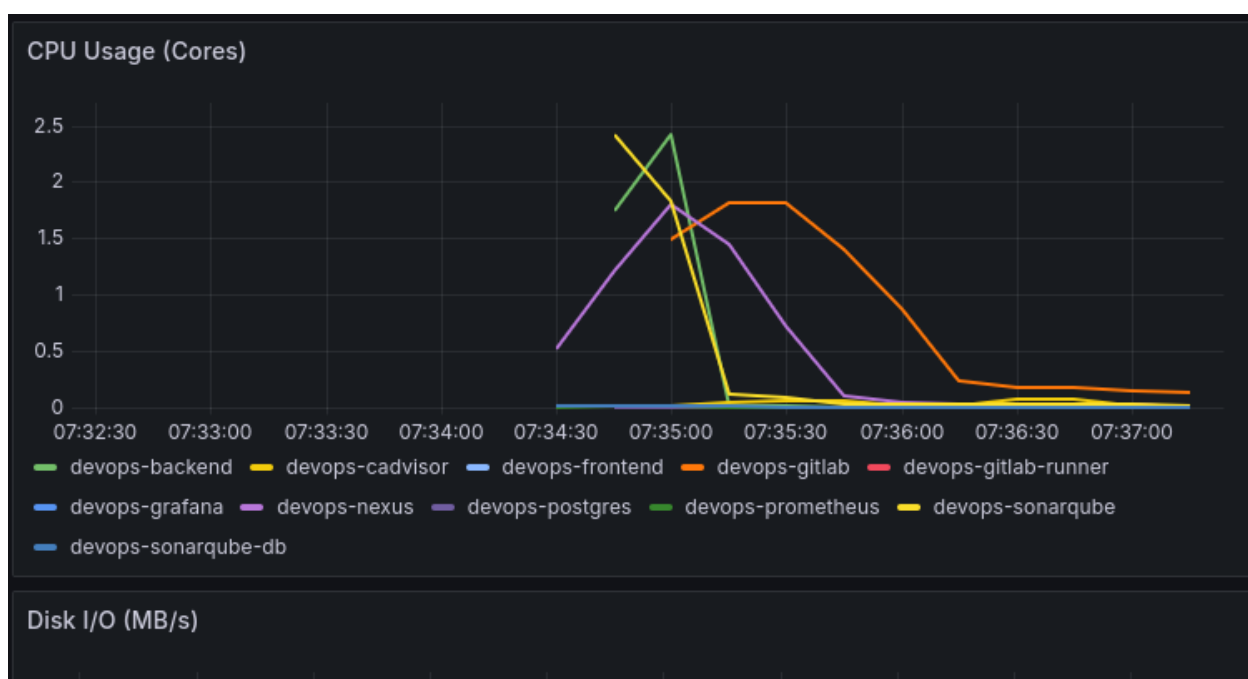


Рисунок 12 — Дашборд в Grafana

2.5 Компоненты обеспечения качества и хранения артефактов

Для обеспечения непрерывного контроля качества кода, централизованного хранения артефактов сборки и безопасного управления конфиденциальными данными в состав платформы интегрированы три специализированных инструмента:

- SonarQube выполняет статический анализ исходного кода с выявлением уязвимостей и ошибок
- Nexus Repository Manager обеспечивает хранение и доставку артефактов сборки
- HashiCorp Vault централизованно управляет секретами и ключами доступа. Совместно эти компоненты формируют замкнутый цикл обеспечения качества и безопасности программного обеспечения на всех этапах CI/CD.

SonarQube Community Edition развёрнут в виде двух связанных контейнеров: собственно сервера анализа и выделенной базы данных PostgreSQL для хранения результатов сканирования и конфигурации. Такое разделение обеспечивает сохранность истории анализов при обновлении версии SonarQube и позволяет масштабировать хранилище независимо от вычислительных ресурсов приложения.

Интеграция SonarQube в CI/CD-конвейер осуществляется через агент сканирования - SonarScanner, запускаемый в отдельной стадии sonar. Анализ выполняется после этапа сборки и модульного тестирования, но до упаковки артефактов.

```
sonarqube-check:
  stage: sonar
  image: localhost:5000/sonar-scanner-cli:12.0.0.3214_8.0.1
  script:
    - sonar-scanner
      -Dsonar.projectKey=vulnerable-python-project
      -Dsonar.sources=.
      -Dsonar.host.url=$SONAR_HOST_URL
      -Dsonar.login=$SONAR_TOKEN
  tags:
    - python
```

Рисунок 13 — Пример шаблона для интеграции Sonarqube

Сканер отправляет исходный код и результаты тестового покрытия на сервер SonarQube, где выполняется статический анализ с учётом настроенных правил.

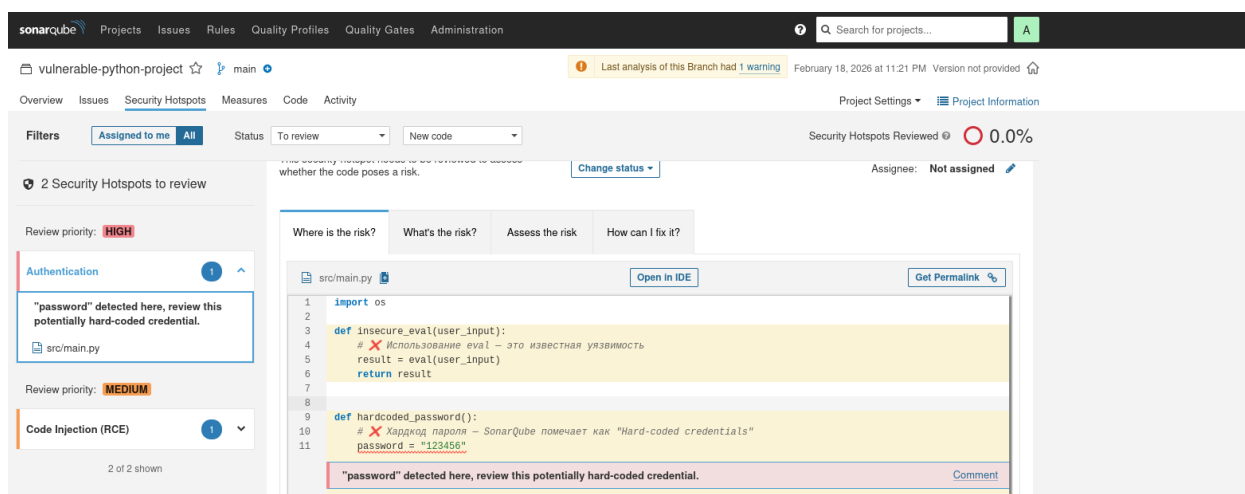


Рисунок 14 — Интерфейс Sonarqube для Python проекта

HashiCorp Vault развёрнут как отдельный сервис инфраструктуры платформы и интегрирован с GitLab CI/CD для безопасного хранения и динамического предоставления конфиденциальных данных. Vault решает задачу централизованного управления секретами, исключая необходимость хранения чувствительной информации в репозиториях кода или системах контроля версий.

В Vault созданы следующие секретные пути для группы проектов платформы:

- SSH-ключи - secret/ssh-keys: приватные SSH-ключи для подключения к целевым серверам деплоя. Ключи монтируются в задания CI/CD при выполнении стадии deploy, обеспечивая безопасный доступ к серверам без необходимости хранения ключей в GitLab.
- Учётные данные Nexus - secret/nexus: логин и пароль сервисной учётной записи для аутентификации в Docker Registry и REST API Nexus. Используются при публикации артефактов и Docker-образов.

Интеграция Vault с GitLab CI/CD реализована через механизм переменных окружения, значения которых извлекаются из Vault непосредственно перед выполнением задания. При запуске конвейера GitLab Runner обращается к Vault по HTTPS, проходит аутентификацию с использованием Token, извлекает необходимые секреты и передаёт их в среду выполнения как переменные окружения. После завершения задания доступ к секретам автоматически отзывается.

Nexus Repository Manager 3 функционирует как центральное хранилище артефактов сборки и Docker-образов. Контейнер exposes два порта: 8081 для веб-интерфейса управления репозиториями и 5000 для взаимодействия с Docker Registry по протоколу Docker HTTP API v2.

Docker Registry на порту 5000 обслуживает два hosted-репозиторий docker-platform, который хранит образы компонентов самой платформы, собираемые при обновлении инфраструктуры. Репозиторий docker-projects, который развернут на 5001 порту предназначен для образов пользовательских приложений — он используется в тех случаях, когда пользователь при создании проекта выбрал опцию контейнеризации.

Репозитории артефактов для конкретных языков программирования используются в сценариях без контейнеризации. Для Java-проектов Maven публикует JAR-файл в maven-releases или maven-snapshots в зависимости от версии. Для C#-проектов утилита dotnet nuget push отправляет пакет в nuget-hosted. Для Python-проектов twine upload публикует wheel-дистрибутив в pypi-hosted. Для JavaScript-проектов npm publish загружает пакет в npm-hosted.

2.6 GitLab CE и GitLab Runner

GitLab Community Edition и GitLab Runner составляют ядро CI/CD-инфраструктуры платформы. GitLab CE выполняет роль системы контроля версий и оркестратора пайплайнов, а GitLab Runner — роль исполнителя задач.

GitLab CE развёрнут в Docker-контейнере, внешний URL установлен как `http://gitlab.local:8929`, порт SSH изменён на 2222. Встроенные Prometheus и Grafana отключены, поскольку платформа использует собственную подсистему мониторинга. Контейнер подключён к сети `devops-network` с DNS-алиасом `gitlab`, что позволяет бэкенду обращаться к API по внутреннему адресу `http://gitlab:80`. Данные сохраняются в трёх именованных томах: конфигурация, журналы и репозитории.

Все проекты пользователей создаются внутри одной группы GitLab, что обеспечивает единое пространство имён и позволяет использовать групповые CI/CD-переменные - `SONAR_HOST_URL`, `SONAR_TOKEN`, `NEXUS_USER`, `NEXUS_PASSWORD`, `SSH_PRIVATE_KEY` и т. д.

GitLab Runner развёрнут в Docker-контейнере на основе образа `gitlab/gitlab-runner:alpine` и подключён к сети `devops-network` (таблица 5). Все Runner используют executor типа `docker` — каждое задание выполняется в изолированном контейнере с образом, загружаемым из локального Nexus Docker Registry.

Таблица 5 - Зарегистрированные GitLab Runner

Имя	Тег	Docker-образ	Назначение
<code>java_build_runner</code>	<code>java</code>	<code>maven:3.9.6-eclipse-temurin-17</code>	Сборка Java-проектов
<code>python_build_runner</code>	<code>python</code>	<code>python:3.11</code>	Сборка Python-проектов

Таблица 5 - Зарегистрированные GitLab Runner (продолжение)

Имя	Тег	Docker-образ	Назначение
dotnet_build_runner	dotnet	dotnet/sdk:8.0	Сборка C#-проектов
runner_for_frontend	node	node:20-alpine	Сборка фронтенда
sonar_scan_runner	sonar	sonar-scanner-cli	Анализ SonarQube
docker_build_runner	docker	docker:24	Сборка Docker-образов (DinD)
ansible_deploy	ansible	cytopia/ansible:2.13	Деплой через Ansible

Джобы – автономные единицы работы или задачи, выполняемые независимо от других подобных задач - в GitLab имеют единую структуру из пяти стадий, представленных в таблице 6.

Таблица 6 - Стадии CI/CD-пайплайнов

Стадия	Назначение
build	Компиляция бэкенда и сборка фронтенда (параллельно)
test	Модульные тесты бэкенда и фронтенда
sonar	Статический анализ SonarQube
push	Публикация артефактов в Nexus
deploy	Развёртывание через Ansible

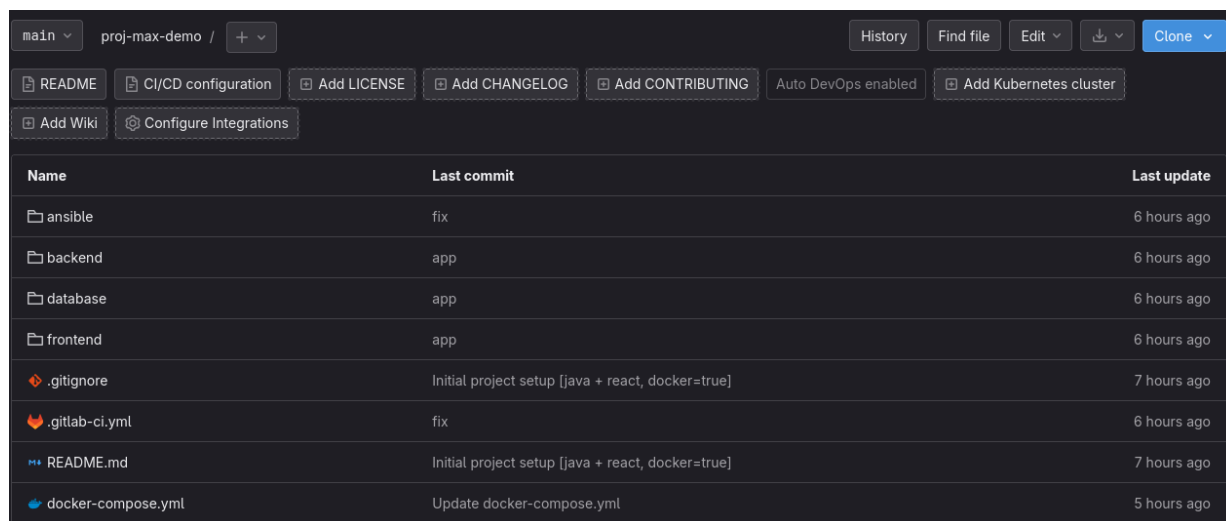
Различия между вариантами конвейеров определяются способом упаковки. В Docker-варианте стадия build собирает Docker-образы через DinD, стадия push отправляет их в Nexus Docker Registry вместе с docker-compose.yml, а стадия deploy выполняет docker pull и docker-compose up на целевом сервере. В варианте без Docker стадия build выполняет стандартную

компиляцию, стадия push упаковывает артефакты в .tar.gz и загружает в Raw-репозиторий Nexus через curl, а стадия deploy скачивает архивы и запускает приложение непосредственно на сервере.

2.7 Проверка работы платформы

Рассмотрим работу платформы (сценарий использования) на примере проекта «task-manager» — одностраничного веб-приложения для управления задачами, реализованного на стеке Java + Angular + PostgreSQL с Docker-контейнеризацией.

После регистрации на платформе и создания проекта с указанным стеком, в GitLab автоматически создаётся репозиторий с начальной структурой файлов, включая .gitlab-ci.yml по шаблону java-angular-docker.yml (рисунок 15).



main	proj-max-demo / +	History	Find file	Edit	Download	Clone
README	CI/CD configuration	Add LICENSE	Add CHANGELOG	Add CONTRIBUTING	Auto DevOps enabled	Add Kubernetes cluster
Add Wiki	Configure Integrations					
Name	Last commit	Last update				
ansible	fix	6 hours ago				
backend	app	6 hours ago				
database	app	6 hours ago				
frontend	app	6 hours ago				
.gitignore	Initial project setup [java + react, docker=true]	7 hours ago				
.gitlab-ci.yml	fix	6 hours ago				
README.md	Initial project setup [java + react, docker=true]	7 hours ago				
docker-compose.yml	Update docker-compose.yml	5 hours ago				

Рисунок 15 — Репозиторий с файлами проекта

Далее, после отправки кода запускается конвейер из четырех стадий – build, sonar, push, deploy.

На стадии build параллельно выполняются три задания: сборка Java-проекта утилитой Maven для последующего анализа SonarQube и сборка

Docker-образов бэкенда и фронтенда через Docker-in-Docker. Каждый образ тегируется уникальным идентификатором, включающим адрес Nexus Docker Registry и номер пайплайна.

На стадии **sonar** SonarScanner отправляет исходный код и скомпилированные классы на сервер SonarQube. По завершении анализа доступен отчёт с метриками качества: количество ошибок, уязвимостей, «запахов кода» и уровень дублирования (рисунок 16).

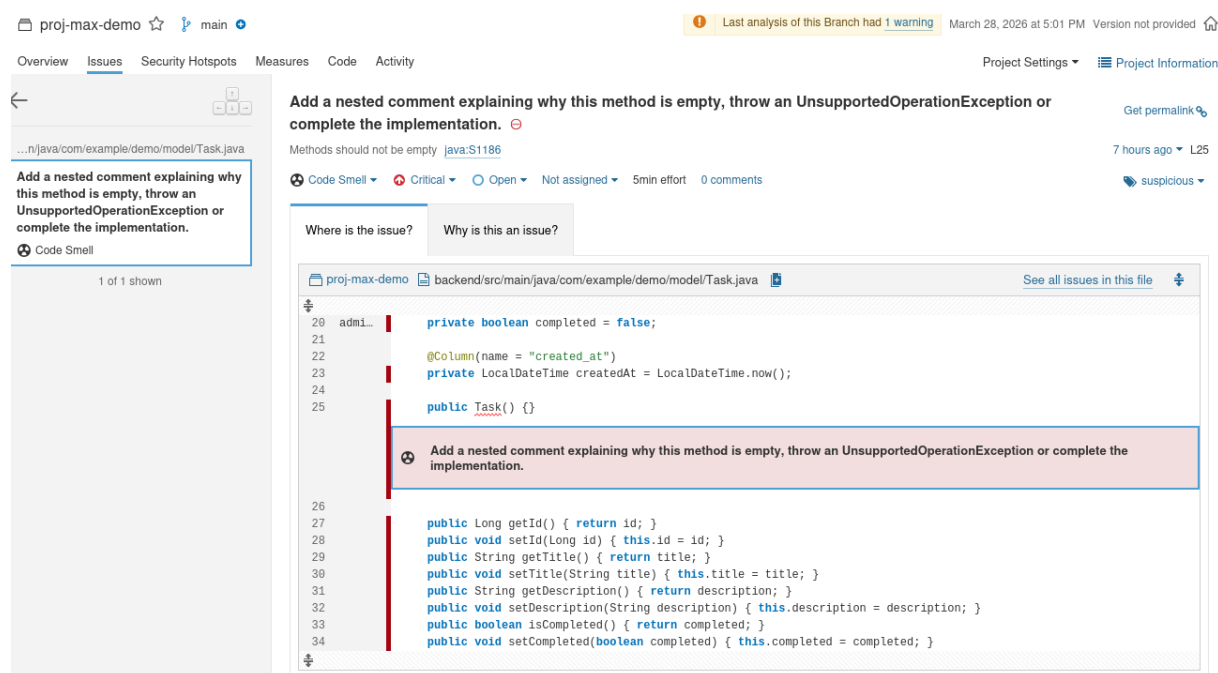


Рисунок 16 — Результат анализа SonarQube для проекта

На стадии **push** Docker-образы бэкенда, фронтенда и базы данных отправляются в Nexus Docker Registry. Дополнительно в Raw-репозиторий Nexus загружается файл `docker-compose.yml` для совместного запуска контейнеров при деплое.

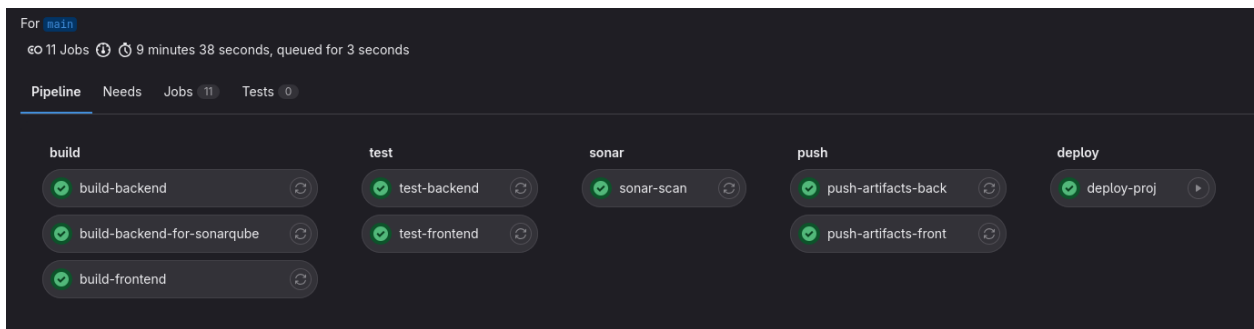


Рисунок 17 — Пайплайн проекта в GitLab

Перед развертыванием следует определять IP-адрес сервера, на котором будет работать приложение: эта информация необходима, чтобы убедиться в том, что приложение действительно работает на целевом сервере (рисунок 18).

```
sys_ansible@deploy-server:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:68:48:34 brd ff:ff:ff:ff:ff:ff
    altname enx080027684834
    inet 192.168.0.103/24 brd 192.168.0.255 scope global dynamic noprefixroute enp0s3
        valid_lft 4622sec preferred_lft 3722sec
    inet6 fe80::662a:1a88:209d:640b/64 scope link
        valid_lft forever preferred_lft forever
```

Рисунок 18 — Адрес целевого сервера

Стадия `deploy` запускается вручную после проверки результатов предыдущих стадий. Runner с образом Ansible подключается к целевому серверу по SSH, скачивает `docker-compose.yml` из Nexus, выполняет `docker login` в Docker Registry, извлекает образы и запускает контейнеры приложения. После завершения деплоя таск-менеджер становится доступен по адресу целевого сервера, что продемонстрировано на рисунке 19.



Task Manager (Docker)

Add Task



Новое задание

Пример работы приложения



123

123



Рисунок 19 — Развернутое приложение на целевом сервере

ГЛАВА 3 Тестирование платформы

3.1 Сравнение ручного и автоматизированного развертывания

Для оценки эффективности разработанной платформы было проведено сравнение двух подходов к развёртыванию веб-приложений: ручного деплоя, выполняемого разработчиком самостоятельно, и автоматизированного деплоя через CI/CD-конвейер платформы. Сравнение проводилось для двух типов проектов: с Docker-контейнеризацией (Java + Angular + PostgreSQL) и без контейнеризации (Python + React + PostgreSQL), что позволило оценить эффективность платформы для различных сценариев использования.

Для каждого подхода и типа проекта фиксировались следующие метрики: общее время от момента готовности кода до работающего приложения на сервере, количество выполняемых вручную операций, количество потенциальных точек отказа. Измерения проводились для пяти последовательных развёртываний каждой конфигурации, что позволило оценить не только абсолютные показатели, но и стабильность процесса.

При ручном подходе разработчик выполняет следующую последовательность действий: сборка бэкенда командой `mvn clean package`, сборка фронтенда командами `npm install` и `npm run build`, сборка Docker-образов для каждого компонента, авторизация в Docker Registry, отправка образов в registry, подключение к серверу по SSH, редактирование файла `docker-compose.yml` с указанием актуальных тегов образов, остановка предыдущей версии приложения, загрузка новых образов и запуск контейнеров.

Для проекта без контейнеризации последовательность действий отличается: установка зависимостей Python, сборка фронтенда, подключение к серверу по SSH, копирование файлов бэкенда и фронтенда через `scp` или `rsync`, настройка виртуального окружения Python на сервере, применение миграций базы данных, настройка и перезапуск `systemd`-сервисов или `Gunicorn`, настройка `Nginx` для раздачи статики фронтенда.

При использовании платформы для обоих типов проектов разработчик выполняет только два действия: отправка кода в репозиторий командой `git push` и подтверждение запуска деплоя нажатием кнопки в интерфейсе GitLab. Все остальные операции выполняются автоматически согласно шаблону конвейера, соответствующему выбранному стеку.

Результаты замеров времени для пяти последовательных развёртываний представлены в таблице 7, для проекта без использования Docker представлены в таблице 8.

Таблица 7 - Время развёртывания Docker-проекта в минутах

Итерация	Ручной деплой	Автоматизированный деплой
1	38	9
2	42	8
3	35	8
4	51	9
5	40	9
Среднее	41.2	8.6
Станд. откл.	5.9	0.5

Таблица 8 - Время развёртывания проекта без Docker в минутах

Итерация	Ручной деплой	Автоматизированный деплой
1	32	6
2	28	7
3	35	6
4	45	7
5	30	6
Среднее	34.0	6.4
Станд. откл.	6.5	0.5

По полученным в ходе тестирования данным, ручное развёртывание проекта без Docker в среднем быстрее, поскольку не требует сборки и

отправки Docker-образов. Однако стандартное отклонение выше, что объясняется большим количеством шагов настройки на сервере и их вариативностью. Автоматизированный деплой проекта без Docker также быстрее, так как отсутствует этап сборки и публикации образов. При этом стабильность обоих автоматизированных подходов одинаково высока. Сравнение времени развёртывания для обоих типов проектов представлено на рисунке 20.

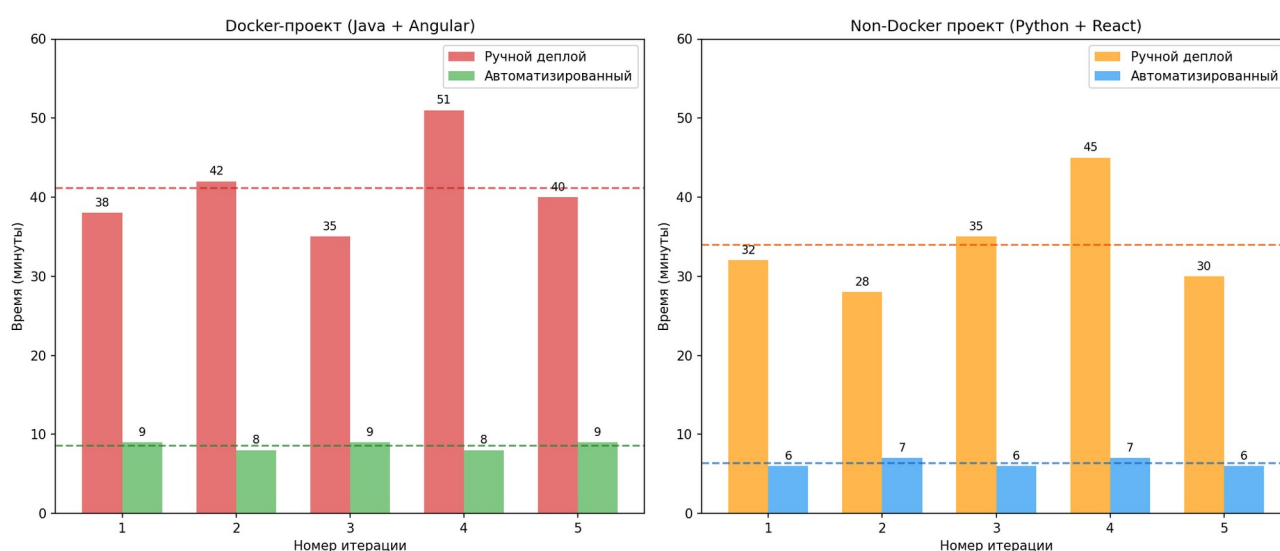


Рисунок 20 — Сравнение времени деплоя для проектов

Сравнительный анализ по ключевым критериям для обоих типов проектов приведён в таблице 9.

Таблица 9 - Сравнение критериев по типу проекта

Критерий	Docker ручной	Docker авто	Non-Docker ручной	Non-Docker авто
Среднее время	41.2 мин	8.6 мин	34.0 мин	6.4 мин
Ручные операции	12–15	2	14–18	2
Вводимые команды	18–22	1	22–28	1
Точки отказа	8–10	2	10–14	2
Коэффициент ускорения	-	4.8x	-	5.3x

Проект без Docker при ручном деплое требует большего количества операций и команд из-за необходимости настройки окружения непосредственно на сервере, копирования файлов, управления процессами через systemd. Однако автоматизированный деплой нивелирует эту разницу: Ansible-плейбук выполняет все шаги одинаково при каждом запуске. Коэффициент ускорения для non-Docker проекта даже выше, поскольку автоматизация устраняет большее количество ручных операций. Распределение времени по этапам для обоих типов проектов показано на рисунке 21.

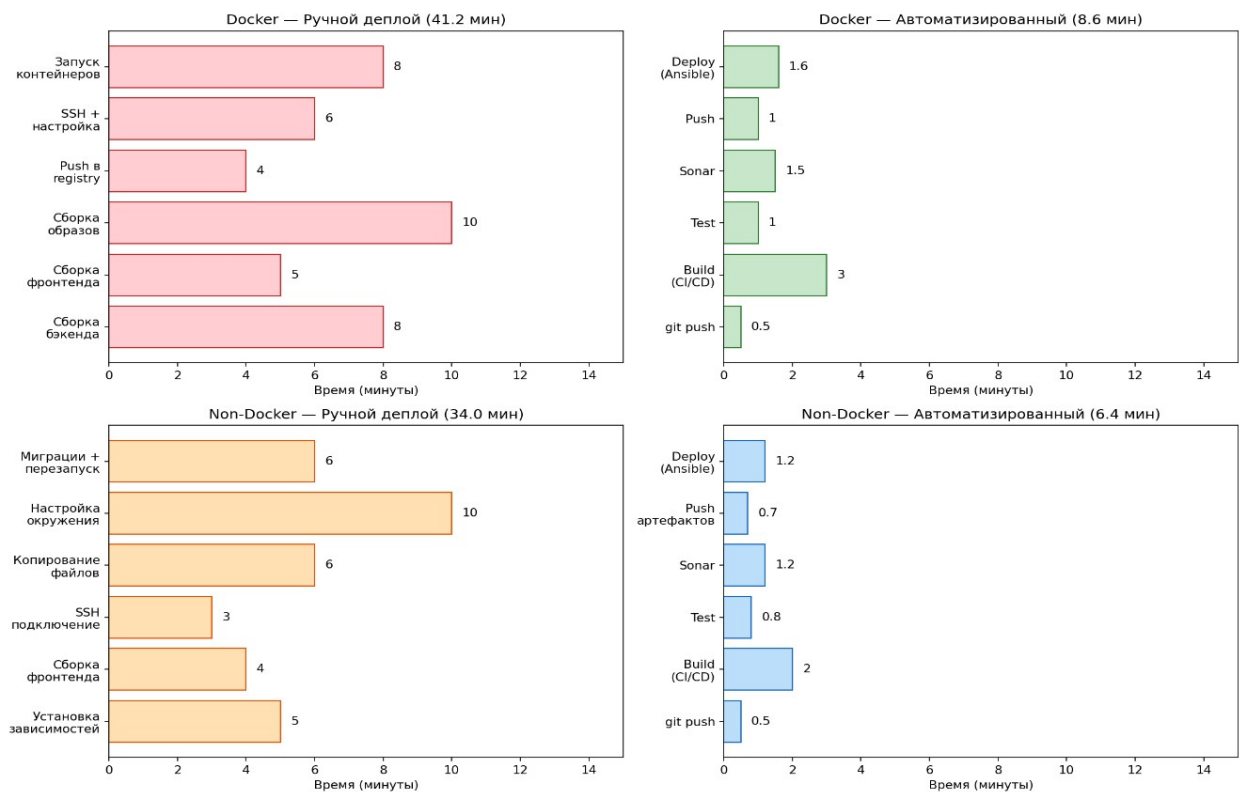


Рисунок 21 - Распределение времени по этапам развёртывания

При оценке масштабирования разница становится более существенной. Зависимость суммарных временных затрат от количества развёртываний в месяц для обоих типов проектов представлена на рисунке 22.

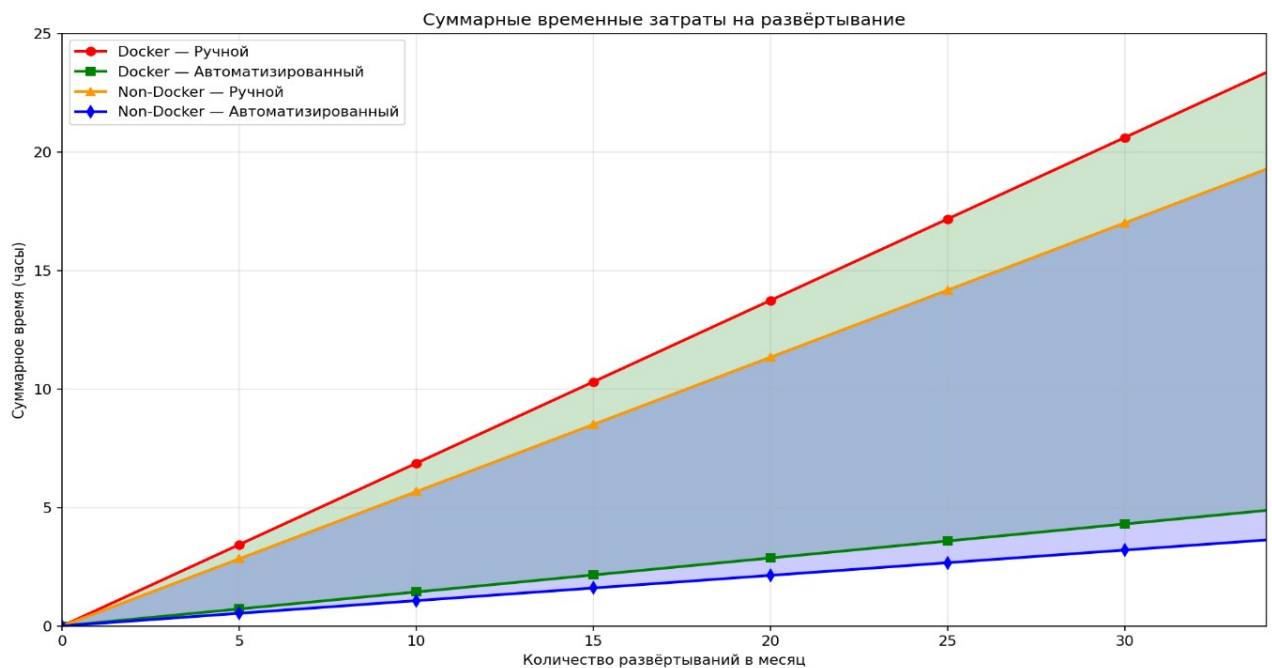
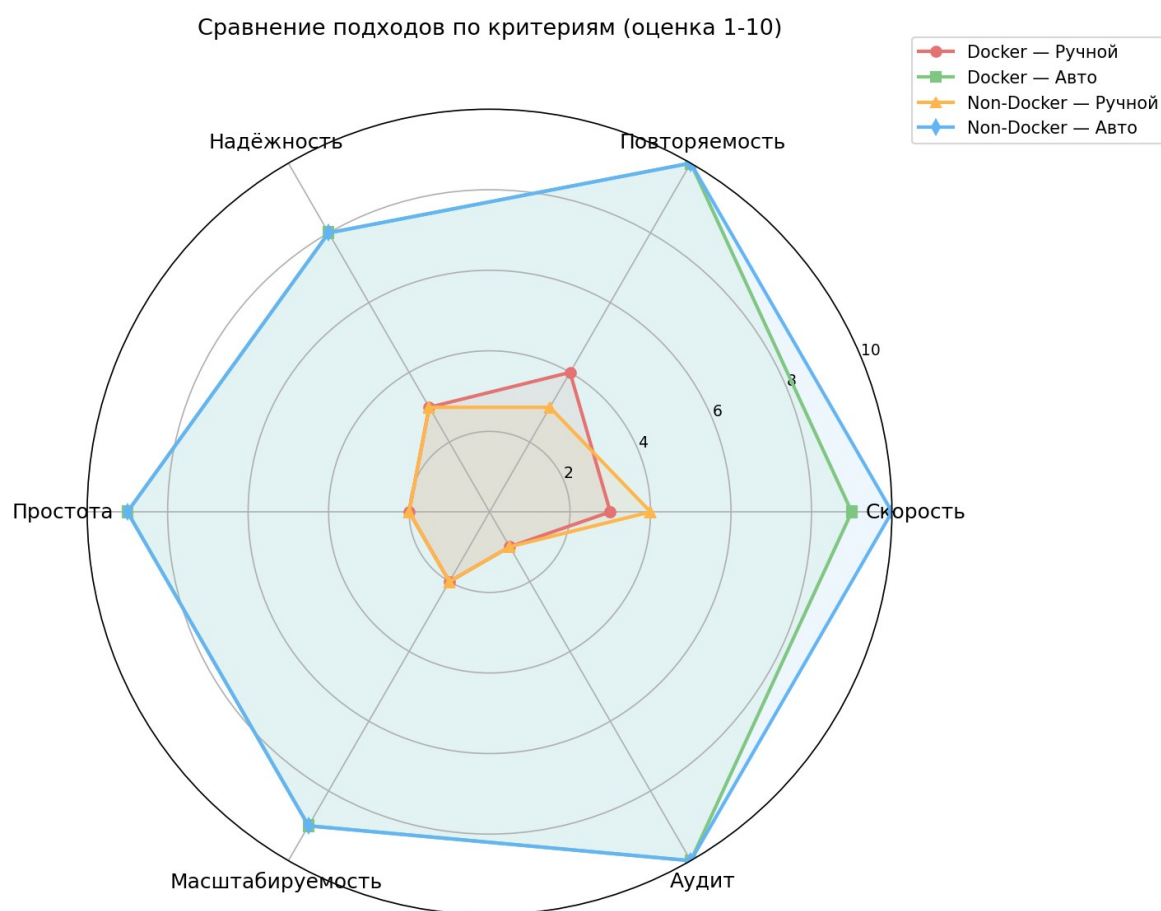


Рисунок 22 - Суммарные временные затраты в зависимости от частоты деплоя

При 20 развёртываниях в месяц экономия времени составляет: для Docker-проекта — 10.8 часов, для non-Docker проекта — 9.2 часа. Годовая экономия для команды, работающей с обоими типами проектов, может достигать 240 часов, что эквивалентно 30 полным рабочим дням. Сравнение подходов по качественным критериям представлено на рисунке 23.



Рис

унок 23 — График сравнения подходов

Таким образом, автоматизированное развёртывание через разработанную платформу обеспечивает сокращение времени деплоя в 4.8–5.3 раза вне зависимости от типа проекта, уменьшение количества ручных операций в 6–9 раз, снижение вероятности ошибок за счёт исключения человеческого фактора, полную повторяемость результата и автоматический аудит всех действий через логи CI/CD-конвейера. Платформа одинаково

эффективна как для контейнеризированных, так и для традиционных приложений.

3.2 Анализ работы платформы в аварийных ситуациях

Для подтверждения предсказуемости поведения платформы в случае возникновения ошибок на различных этапах CI/CD-пайплайна были смоделированы четыре аварийных сценария, охватывающие типичные проблемы разработки: уязвимости в коде, падающие тесты, ошибки конфигурации сборки и ошибки компиляции.

Первый сценарий направлен на проверку работы статического анализа кода. В Java-бэкенд проекта были намеренно внесены три типа проблем: хардкод пароля непосредственно в коде контроллера, формирование SQL-запроса через конкатенацию строк без использования параметризованных запросов, а также использование `System.out.println` вместо логгера. После отправки кода в репозиторий и запуска конвейера на стадии sonar SonarScanner проанализировал исходный код и передал отчёт на сервер SonarQube. Обнаружение одной из ошибок изображено на рисунке 24.

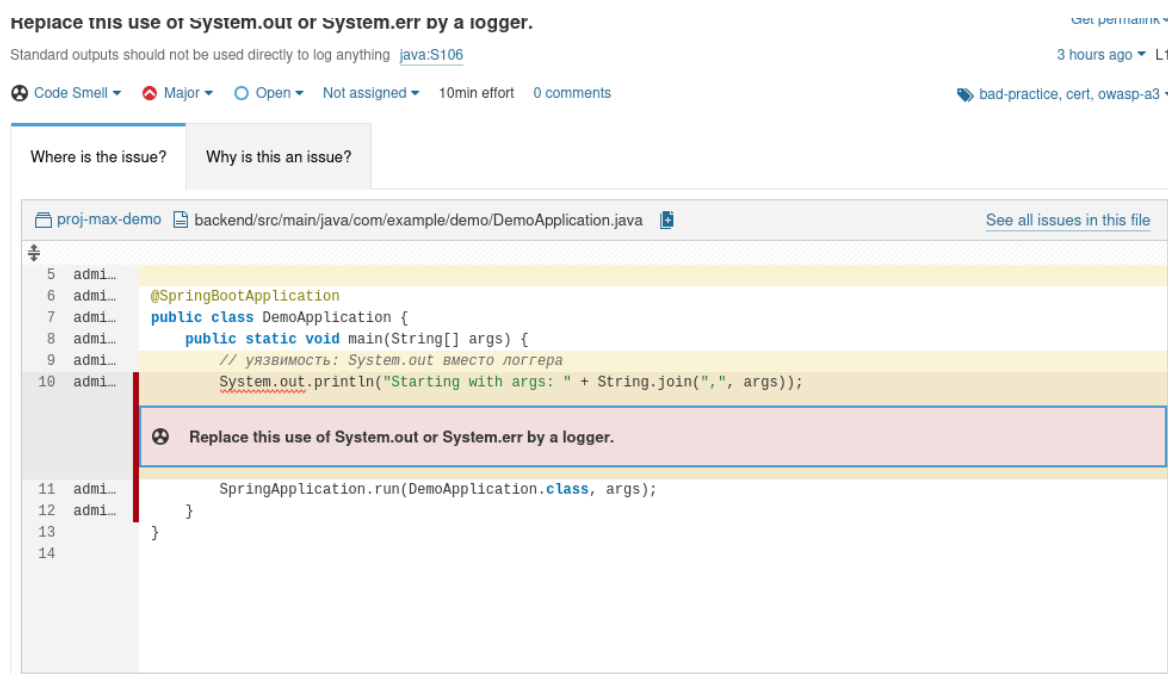


Рисунок 24 — Отчет по уязвимости кода

Система статического анализа корректно идентифицировала все внесённые проблемы: хардкод секретов был классифицирован как критическая уязвимость безопасности, небезопасный SQL-запрос — как потенциальная SQL-инъекция, использование `System.out.println` — как нарушение практик логирования. Quality Gate не был пройден, и благодаря настройке `-Dsonar.qualitygate.wait=true` пайплайн завершился с ошибкой, прерванный пайплайн изображен на рисунке 25.

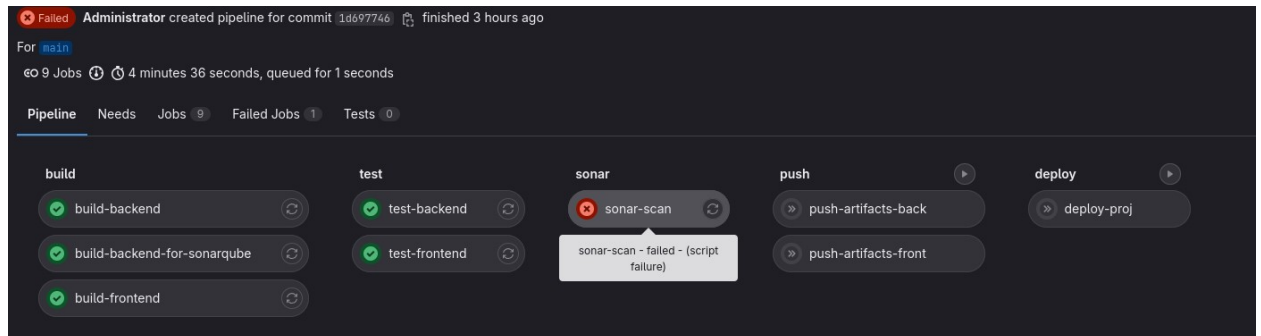


Рисунок 25 — Прерванный пайплайн на стадии sonar

Второй сценарий проверял реакцию платформы на падающие модульные тесты. В тестовый класс Java-проекта были добавлены три заведомо теста: тест с неверным утверждением, тест, вызывающий `NullPointerException` при обращении к методу на null-объекте, и тест, ожидающий выброс исключения, которое не генерируется тестируемым методом. На стадии `test-backend` фреймворк JUnit выполнил все тесты и зафиксировал три провала с исключениями `AssertionError` и `NullPointerException`. Логи задания содержат детальную информацию о каждом упавшем тесте с указанием ожидаемого и фактического значений. Данный сценарий демонстрирует, что платформа корректно фиксирует проблемы на этапе тестирования и предоставляет разработчику полную диагностическую информацию для исправления ошибок. Результат выполнения тестов с ошибками показан на рисунке 26.

```
DevOps Platform Projects > max-demo > Jobs > #766

Showing last 500.00 KIB of log - Complete Raw

4798 2026-04-10T22:05:51.892827Z 010 [ERROR] Errors:
4799 2026-04-10T22:05:51.892832Z 010 [ERROR] TaskRepositoryTest.testUnsafeQueryNullRepo:15 NullPointerException Cannot invoke "com.example.demo.repository.TaskRepository.findTasksByUnsafeTitle(String)" because "repo" is null
4800 2026-04-10T22:05:51.892896Z 010 [INFO]
4801 2026-04-10T22:05:51.893008Z 010 [ERROR] Tests run: 4, Failures: 2, Errors: 1, Skipped: 0
4802 2026-04-10T22:05:51.893014Z 010 [INFO]
4803 2026-04-10T22:05:51.896857Z 010 [INFO] -----
4804 2026-04-10T22:05:51.896869Z 010 [INFO] BUILD FAILURE
4805 2026-04-10T22:05:51.896878Z 010 [INFO] -----
4806 2026-04-10T22:05:51.904732Z 010 [INFO] Total time: 43.152 s
4807 2026-04-10T22:05:51.905025Z 010 [INFO] Finished at: 2026-04-10T22:05:51Z
4808 2026-04-10T22:05:51.905162Z 010 [INFO] -----
4809 2026-04-10T22:05:51.906822Z 010 [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.1.2:test (default-test) on project demo-docker-app: There are test failures.
4810 2026-04-10T22:05:51.906834Z 010 [ERROR]
4811 2026-04-10T22:05:51.906892Z 010 [ERROR] Please refer to /builds/devops-projects/proj-max-demo/backend/target/surefire-reports for the individual test results.
4812 2026-04-10T22:05:51.907106Z 010 [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream.
4813 2026-04-10T22:05:51.907151Z 010 [ERROR] -> [Help 1]
4814 2026-04-10T22:05:51.907236Z 010 [ERROR]
4815 2026-04-10T22:05:51.907470Z 010 [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
4816 2026-04-10T22:05:51.907527Z 010 [ERROR] Re-run Maven using the -X switch to enable full debug logging.
4817 2026-04-10T22:05:51.907609Z 010 [ERROR]
4818 2026-04-10T22:05:51.907614Z 010 [ERROR] For more information about the errors and possible solutions, please read the following articles:
4819 2026-04-10T22:05:51.907713Z 010 [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
4820 2026-04-10T22:05:52.215837Z 000 section_end:1775858752:step_script
4821 2026-04-10T22:05:52.215851Z 000+section_start:1775858752:cleanup_file_variables
4822 2026-04-10T22:05:52.216891Z 000+Cleaning up project directory and file based variables
4823 2026-04-10T22:05:52.774356Z 000 section_end:1775858752:cleanup_file_variables
4824 2026-04-10T22:05:52.774367Z 000+
4825 2026-04-10T22:05:52.837216Z 000 ERROR: Job failed: exit code 1
```

Рисунок 26 — Проваленные тесты

Третий сценарий моделировал ошибку конфигурации сборки фронтенда. В Dockerfile фронтенд-приложения была указана команда `npm run build:prod`, которая отсутствует в файле `package.json` проекта. Сам Dockerfile синтаксически корректен и проходит валидацию, однако при выполнении команды `RUN npm run build:prod` на этапе сборки образа `npm` возвращает ошибку `missing script: build:prod`. Docker-демон фиксирует ненулевой код возврата и прерывает сборку образа. Задание `build-frontend` переходит в статус `Failed`, образ не создаётся и не отправляется в Nexus Docker Registry. Последующие стадии публикации и деплоя не запускаются. Этот сценарий показывает, что платформа обнаруживает не только синтаксические ошибки конфигурации, но и логические несоответствия между Dockerfile и структурой проекта. Лог сборки Docker с ошибкой `npm` представлен на рисунке 27.


```
DevOps Platform Projects > max-demo > Jobs > #774

Search job log [Q] [P] [D] [U] [D]

189 2026-04-10T22:05:52.288471Z 01E #12 [build 6/6] RUN npm run build:prod
190 2026-04-10T22:05:52.765361Z 01E #12 0.628 npm error Missing script: "build:prod"
191 2026-04-10T22:05:52.765681Z 01E #12 0.628 npm error
192 2026-04-10T22:05:52.765687Z 01E #12 0.628 npm error To see a list of scripts, run:
193 2026-04-10T22:05:52.765693Z 01E #12 0.628 npm error npm run
194 2026-04-10T22:05:52.813681Z 01E #12 0.634 npm error A complete log of this run can be found in: /root/.npm/_logs/2026-04-10T22_05_52_593Z-debu
g-0.log
195 2026-04-10T22:05:52.813733Z 01E #12 ERROR: process "/bin/sh -c npm run build:prod" did not complete successfully: exit code: 1
196 2026-04-10T22:05:52.813740Z 01E -----
197 2026-04-10T22:05:52.813745Z 01E > [build 6/6] RUN npm run build:prod:
198 2026-04-10T22:05:52.813750Z 01E 0.628 npm error Missing script: "build:prod"
199 2026-04-10T22:05:52.813755Z 01E 0.628 npm error
200 2026-04-10T22:05:52.813758Z 01E 0.628 npm error To see a list of scripts, run:
201 2026-04-10T22:05:52.813764Z 01E 0.628 npm error npm run
202 2026-04-10T22:05:52.813768Z 01E 0.634 npm error A complete log of this run can be found in: /root/.npm/_logs/2026-04-10T22_05_52_593Z-debug-0.log
203 2026-04-10T22:05:52.813773Z 01E -----
204 2026-04-10T22:05:52.814950Z 01E Dockerfile:7
205 2026-04-10T22:05:52.814976Z 01E -----
206 2026-04-10T22:05:52.815010Z 01E 5 | COPY . .
207 2026-04-10T22:05:52.815015Z 01E 6 | # Ошибка: запускаем несуществующий скрипт
208 2026-04-10T22:05:52.815372Z 01E 7 | >>> RUN npm run build:prod
209 2026-04-10T22:05:52.815378Z 01E 8 |
210 2026-04-10T22:05:52.815382Z 01E 9 | FROM nginx:alpine
211 2026-04-10T22:05:52.815386Z 01E -----
212 2026-04-10T22:05:52.815388Z 01E ERROR: failed to solve: process "/bin/sh -c npm run build:prod" did not complete successfully: exit code: 1
213 2026-04-10T22:05:53.057144Z 000 section_end:1775858753:step_script
214 2026-04-10T22:05:53.057155Z 000+section_start:1775858753:cleanup_file_variables
215 2026-04-10T22:05:53.057887Z 000+Cleaning up project directory and file based variables
216 2026-04-10T22:05:53.576595Z 000 section_end:1775858753:cleanup_file_variables
217 2026-04-10T22:05:53.576611Z 000+
218 2026-04-10T22:05:56.488140Z 000 ERROR: Job failed: exit code 1
```

Рисунок 27 — Ошибка при сборке Dockerfile

Четвёртый сценарий был направлен на проверку обработки ошибок компиляции. В C#-проекте в классе TaskItem.cs было объявлено два свойства с одинаковым именем Title, что является нарушением правил языка. При запуске стадии build-backend компилятор .NET обнаружил конфликт и вернул ошибку CS0102. Сборка была прервана, задание завершилось с кодом возврата 1. Пайплайн перешёл в статус Failed, и все последующие стадии не были инициированы. Платформа корректно обработала ошибку компиляции и предотвратила дальнейшее распространение нерабочего кода. Лог компиляции .NET с ошибкой представлен на рисунке 28.

Проведённые аварийные тесты подтвердили, что платформа обеспечивает многоуровневую защиту от развёртывания дефектного кода. Статический анализ SonarQube выявляет уязвимости безопасности и нарушения стандартов кодирования до этапа публикации. Модульные тесты фиксируют логические ошибки в бизнес-логике приложения. Сборка

```
DevOps Platform Projects > homelander-taskmanager > Jobs > #790

38 2026-04-10T22:11:00.910730Z 010 An error occurred while sending the request.
39 2026-04-10T22:11:30.510763Z 010 Unable to read data from the transport connection: Connection reset by peer.
40 2026-04-10T22:11:30.510768Z 010 Connection reset by peer
41 2026-04-10T22:11:30.880269Z 010 Retrying 'FindPackagesByIdAsync' for source 'https://api.nuget.org/v3-flatcontainer/microsoft.extensions.logging.abstractions/index.json'.
42 2026-04-10T22:11:30.880284Z 010 An error occurred while sending the request.
43 2026-04-10T22:11:30.880297Z 010 Unable to read data from the transport connection: Connection reset by peer.
44 2026-04-10T22:11:30.880359Z 010 Connection reset by peer
45 2026-04-10T22:11:33.609915Z 010 Retrying 'FindPackagesByIdAsync' for source 'https://api.nuget.org/v3-flatcontainer/microsoft.entityframeworkcore.relational/index.json'.
46 2026-04-10T22:11:33.609928Z 010 An error occurred while sending the request.
47 2026-04-10T22:11:33.609934Z 010 Unable to read data from the transport connection: Connection reset by peer.
48 2026-04-10T22:11:33.609938Z 010 Connection reset by peer
49 2026-04-10T22:11:40.273315Z 010 Failed to download package 'System.Composition.Runtime.6.0.0' from 'https://api.nuget.org/v3-flatcontainer/system.composition.runtime/6.0.0/system.composition.runtime.6.0.0.nupkg'.
50 2026-04-10T22:11:40.273326Z 010 An error occurred while sending the request.
51 2026-04-10T22:11:40.273331Z 010 Unable to read data from the transport connection: Connection reset by peer.
52 2026-04-10T22:11:40.273334Z 010 Connection reset by peer
53 2026-04-10T22:12:52.479721Z 010 Restored /builds/devops-projects/proj-homelander-taskmanager/backend/project.csproj (in 1.41 min).
54 2026-04-10T22:12:52.514734Z 010 $ dotnet publish -c Release -o publish
55 2026-04-10T22:12:53.132544Z 010 Determining projects to restore...
56 2026-04-10T22:12:53.533918Z 010 All projects are up-to-date for restore.
57 2026-04-10T22:12:55.828191Z 010 /builds/devops-projects/proj-homelander-taskmanager/backend/Models/TaskItem.cs(19,19): error CS0102: The type 'TaskItem' already contains a definition for 'Title' [/builds/devops-projects/proj-homelander-taskmanager/backend/project.csproj]
58 2026-04-10T22:12:55.828202Z 010 /builds/devops-projects/proj-homelander-taskmanager/backend/Models/TaskItem.cs(26,19): error CS0102: The type 'TaskItem' already contains a definition for 'Title' [/builds/devops-projects/proj-homelander-taskmanager/backend/project.csproj]
59 2026-04-10T22:12:56.152207Z 000 section_end:1775859176:step_script
60 2026-04-10T22:12:56.152216Z 000+section_start:1775859176:cleanup_file_variables
61 2026-04-10T22:12:56.152920Z 000+Cleaning up project directory and file based variables
62 2026-04-10T22:12:56.458228Z 000 section_end:1775859176:cleanup_file_variables
63 2026-04-10T22:12:56.458237Z 000+
64 2026-04-10T22:12:56.555603Z 000 ERROR: Job failed: exit code 1
```

Рисунок 28 — Ошибка компиляции

Docker-образов обнаруживает несоответствия конфигурации и зависимостей. Компиляция исходного кода блокирует синтаксические и семантические ошибки на самом раннем этапе. Каждый из этих механизмов автоматически останавливает пайплайн при обнаружении проблемы, исключая необходимость ручного контроля и предотвращая попадание нерабочего или небезопасного кода в продакшен.

3.3 Варианты улучшения и развития платформы

Несмотря на то, что разработанная платформа уже обеспечивает автоматизацию процессов сборки, тестирования и развертывания приложений, её архитектура допускает дальнейшее расширение и улучшение, направленные на повышение масштабируемости, надёжности, безопасности и уровня автоматизации.

Оркестрация контейнеров с использованием Kubernetes - в текущей реализации для управления контейнерами используется Docker Compose, что

является эффективным решением для небольших инфраструктур. Однако при увеличении нагрузки и количества сервисов целесообразно перейти к использованию Kubernetes.

Kubernetes предоставляет более продвинутые механизмы управления контейнеризированными приложениями:

- автоматическое масштабирование (Horizontal Pod Autoscaler);
- самовосстановление контейнеров (перезапуск при сбоях);
- балансировка нагрузки между экземплярами сервисов;
- управление конфигурациями и секретами через ConfigMap и Secret.

Переход на Kubernetes позволит:

- обеспечить высокую доступность платформы;
- масштабировать отдельные компоненты независимо друг от друга;
- упростить управление инфраструктурой при росте количества пользователей.

Улучшение системы мониторинга - существующая подсистема мониторинга на базе Prometheus и Grafana обеспечивает сбор и визуализацию метрик, однако не реализует механизм оперативного уведомления о проблемах.

Для повышения наблюдаемости платформы предлагается внедрение системы алертинга с использованием Alertmanager, входящего в экосистему Prometheus. Alertmanager позволяет:

- задавать правила оповещений (например, высокая загрузка CPU, недоступность сервиса);
- группировать и фильтровать уведомления;
- отправлять сообщения во внешние системы.

В качестве канала уведомлений может быть использован Telegram. Это позволит администраторам оперативно получать информацию о сбоях в режиме реального времени.

Примеры сценариев алертинга:

- недоступность бэкенда более 1 минуты;
- превышение использования памяти контейнером более 90%;
- падение CI/CD-пайплайна.

Интеграция Telegram-бота с Alertmanager обеспечит удобный и быстрый канал реагирования на инциденты.

Использование GitLab Runner Autoscaler - в текущей архитектуре GitLab Runner работает в статическом режиме, что означает фиксированное количество доступных исполнителей задач. При увеличении числа пользователей и параллельных пайплайнов это может привести к очередям и увеличению времени выполнения задач.

Для решения данной проблемы предлагается внедрение механизма GitLab Runner Autoscaler. Он позволяет динамически создавать и удалять раннеры в зависимости от нагрузки.

Основные преимущества:

- автоматическое масштабирование вычислительных ресурсов;
- снижение времени ожидания выполнения пайплайнов;
- эффективное использование ресурсов (раннеры создаются только при необходимости).

Autoscaler может быть интегрирован с облачными провайдерами или использовать Docker Machine для динамического создания виртуальных машин или контейнеров.

Расширение DevSecOps - в текущей реализации используется статический анализ кода через SonarQube, однако его можно дополнить более специализированными инструментами.

1. SAST (Static Application Security Testing)

Позволяет выявлять уязвимости на этапе анализа исходного кода. В GitLab CI/CD уже предусмотрены встроенные шаблоны для SAST, которые можно интегрировать в существующие пайплайны.

2. DAST (Dynamic Application Security Testing)

Проводит анализ уже развернутого приложения, имитируя атаки на веб-интерфейс. Это позволяет выявить уязвимости, которые невозможно обнаружить статическим анализом (например, XSS, CSRF).

3. Сканирование Docker-образов с помощью Trivy

Инструмент Trivy позволяет анализировать Docker-образы на наличие известных уязвимостей в зависимостях и базовых образах.

Предложенные направления развития позволяют существенно повысить уровень зрелости платформы:

- Kubernetes обеспечивает масштабируемость и отказоустойчивость;
- алертинг через Telegram повышает оперативность реагирования;
- Autoscaler оптимизирует использование ресурсов CI/CD;
- расширенные механизмы безопасности минимизируют риски эксплуатации уязвимостей.

Комплексная реализация данных улучшений позволит трансформировать платформу в полнофункциональное промышленное DevOps-решение, готовое к эксплуатации в условиях высокой нагрузки и требований к безопасности.

ЗАКЛЮЧЕНИЕ

В работе поставлена и решена задача проектирования и программной реализации платформы управления программными проектами, выступающей в роли оркестратора над существующим CI/CD-движком. Платформа поддерживает языки программирования Java, C#, Python.

Разработанная структура взаимодействия компонентов системы, включающей пользовательский интерфейс, серверную часть для обработки логики, базу данных для хранения метаданных о проектах и модули интеграции с внешними сервисами, позволят скрывать сложность инфраструктурной настройки от разработчика, предоставляя единый интерфейс для управления жизненным циклом приложения.

Реализовано централизованное управление зависимостями и артефактами, обеспечена интеграция с репозиторием артефактов для хранения результатов сборки. Предложен и реализован механизм шаблонизации и генерации конфигураций, системы контроля версий и взаимодействие с API GitLab.

Система представляет собой готовое комплексное решение и может быть использована как небольшими командами разработчиков, так и в промышленном создании ПО.

ЛИТЕРАТУРА

1. Баланов, А. Н. DevOps: интеграция и автоматизация : учебное пособие для вузов / А. Н. Баланов. — 2-е изд., стер. — Санкт-Петербург : Лань, 2025. — 240 с. — ISBN 978-5-507-50491-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/440162> (дата обращения: 26.11.2025). — Режим доступа: для авториз. пользователей.
2. Грувер, Г. Запуск и масштабирование DevOps на предприятии / Г. Грувер. — Москва : ДМК Пресс, 2018. — 80 с. — ISBN 978-5-97060-704-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/116130> (дата обращения: 12.02.2026). — Режим доступа: для авториз. Пользователей.
3. Ластер, Б. Jenkins 2: приступаем к работе: перевод с английского / Б.Ластер. — Москва : ДМК Пресс, 2019. — 652 с.